Image

Plateforme de Gestion des Contrats de Maintenance

SolidWall Consulting

Rapport de Projet de Fin d'Études

Pour l'obtention du diplôme de

Licence Appliquée en Informatique

Réalisé par :

Rihab CHOUROU
Eya GHARBI

Encadré par :

Encadrant Académique : [Nom]
Encadrant Professionnel : [Nom]

Année Universitaire 2025-2026

Table des matières

Introduction générale 1
Webographie 2

1 Cadre général et méthodologie **3**
Plan 3
1 Présentation de l'organisme d'accueil 4
1.1 Services de l'organisme 4
1.2 Organigramme de l'organisme 5
2 Cadre du projet 5
2.1 Contexte du projet 5
2.2 Analyse de l'existant 5
2.2.1 Etude de l'existant 5
2.2.2 Critique de l'existant 8
2.2.3 Solution proposée 9
3 Méthodologie adaptée 10
3.0.1 Méthodologie traditionnelle 10
3.0.2 Méthodologie Agile 10
3.0.3 Les douze principes de l'Agile 10
3.0.4 Les valeurs agiles 11
3.1 Les principales méthodes Agiles 12
3.1.1 Choix de la méthode de travail 12
3.2 Fondements de SCRUM 13
3.2.1 Pertinence de SCRUM 13
3.2.2 Rôles de SCRUM 13
3.2.3 Evènements Scrum 14
3.2.4 Artéfacts Scrum 14
3.2.5 Cycle de vie 14
3.3 Langage de modélisation UML – Unified Modeling Language 15
4 Planning prévisionnel (Diagramme de Gantt) 15

2 Analyse et spécifications des besoins **17**
Plan 17
1 Spécification des besoins 18
1.1 Identification des acteurs 18
1.2 Identification des besoins fonctionnels 18
1.2.1 Besoins fonctionnels de l'acteur « Client » 19

1.3 Identification des besoins non fonctionnels 20
2 Modélisation des besoins 22
3 Pilotage du projet avec Scrum 22
3.1 Rôles de SCRUM 23
3.1.1 Backlog de produit 23
3.2 Planification des sprints 36
3.2.1 Sprint 1 - Gestion des Utilisateurs et Authentification 37
3.2.2 Sprint 2 - Gestion des Clients et Contrats 37
3.2.3 Sprint 3 - Gestion des Demandes et Tickets 38
3.2.4 Sprint 4 - Collaboration, Documents et IA 38
4 Environnement de Développement 39
4.1 Environnement matériel 39
4.2 Environnement logiciel 39
4.2.1 Langage de programmation et framework 39
4.2.2 Outils de développement et modélisation 40
4.2.3 Systèmes de Gestion de Bases de données 40
4.2.4 Outil de Gestion de Projet Scrum 41
4.2.5 Environnement de composition de documents 41
4.2.6 Outil de design graphique 41
5 Architecture de l'application 41
5.1 Architecture physique 41
5.2 Architecture logique 42

3 Sprint 1 : Gestion des Utilisateurs et Authentification 43
Plan 43
1 Analyse 50
1.1 Diagramme des cas d'utilisation du premier sprint 50
1.2 Description textuelle de cas d'utilisation « Changer mot de passe » 51
1.3 Description des cas d'utilisation « Gérer utilisateurs » 51
1.3.1 Raffinement de cas d'utilisation « Gérer utilisateurs » 51
1.3.2 Description textuelle de cas d'utilisation « Consulter liste utilisateurs » 52
1.3.3 Description textuelle de cas d'utilisation « Ajouter utilisateur » . . 53
1.3.4 Description textuelle de cas d'utilisation « Modifier utilisateur » . 53
1.3.5 Description textuelle de cas d'utilisation « Désactiver un utilisateur » 54
1.4 Description des cas d'utilisation « Gérer profil » 54
1.4.1 Raffinement de cas d'utilisation « Gérer Profil » 54
1.4.2 Description textuelle de cas d'utilisation « Consulter Profil » . . . 55
1.4.3 Description textuelle de cas d'utilisation « Modifier Profil » 55
2 Conception 56

2.1 Diagrammes de séquence 56
2.2 Modèle de données 56
3 Réalisation 57
3.1 Architecture technique 57
3.2 Interfaces utilisateur 57
4 Tests 58
4.1 Tests unitaires 58
4.2 Tests d'intégration 58

4 Sprint 2 : Gestion des Clients et Contrats 59
Plan 59
1 Analyse 63
1.1 Diagramme des cas d'utilisation du Sprint 2 63
1.2 Description textuelle des cas d'utilisation 63
2 Conception 64
2.1 Diagrammes de séquence 64
2.2 Modèle de données 64
3 Réalisation 65
3.1 Architecture technique 65
3.2 Interfaces utilisateur 65
4 Tests 66
4.1 Tests unitaires 66
4.2 Tests d'intégration 66

5 Sprint 3 : Gestion des Demandes et Tickets 67
Plan 67
1 Analyse 72
1.1 Diagramme des cas d'utilisation du Sprint 3 72
1.2 Description textuelle des cas d'utilisation 72
2 Conception 73
2.1 Diagrammes de séquence 73
2.2 Modèle de données 73
3 Réalisation 74
3.1 Architecture technique 74
3.2 Interfaces utilisateur 74
4 Tests 75
4.1 Tests unitaires 75
4.2 Tests d'intégration 75

6 Sprint 4 : Collaboration, Documents et IA **76**
Plan 76
1 Analyse 82
1.1 Diagramme des cas d'utilisation du Sprint 4 82
1.2 Description textuelle des cas d'utilisation 82
2 Conception 83
2.1 Diagrammes de séquence 83
2.2 Modèle de données 83
3 Réalisation 84
3.1 Architecture technique 84
3.2 Interfaces utilisateur 84
4 Tests 85
4.1 Tests unitaires 85
4.2 Tests d'intégration 85

Introduction générale

La gestion efficace des contrats de maintenance et le suivi rigoureux des interventions constituent des processus stratégiques essentiels à la qualité de service et à la relation client.

En effet, on constate un manque de visibilité, une désorganisation des équipes ainsi qu'un risque de perte d'information qui deviennent de plus en plus prononcés, surtout dans un environnement où les communications s'appuient fréquemment sur des outils variés. Cette dispersion nuit à la traçabilité des actions et à la coordination entre les différentes parties prenantes.

C'est dans ce contexte que se situe notre projet de fin d'études (PFE), effectué au sein de la société SolidWall Consulting. L'objectif est de développer une plateforme centralisée de gestion des contrats de maintenance permettant d'automatiser, structurer et tracer l'ensemble des processus liés aux projets de maintenance logicielle.

Nous sommes convaincus que cette plateforme sera un outil indispensable pour l'entreprise, offrant des fonctionnalités pratiques pour faciliter la gestion de plusieurs projets logiciels pour différents clients.

Structure du rapport :

La structure de notre rapport se décline en quatre chapitres distincts :

- **Le premier chapitre** présente le cadre général du projet, l'organisme d'accueil, l'étude de l'existant et la méthodologie adoptée (Scrum).
- **Le deuxième chapitre** détaille l'analyse et la spécification des besoins fonctionnels et non fonctionnels, ainsi que l'environnement de développement.
- **Le troisième chapitre** décrit l'implémentation du premier sprint, incluant la conception et la réalisation des fonctionnalités de gestion des utilisateurs.
- **Le quatrième chapitre** présente l'implémentation des sprints suivants et les fonctionnalités avancées de la plateforme.

Webographie

1. **ManageEngine**, Documentation officielle - https://www.manageengine.com
2. **Jira Software**, Guide utilisateur - https://www.atlassian.com/software/jira
3. **NexoraTech**, Solutions ERP - https://www.nexoratech.com
4. **NestJS**, Documentation - https://docs.nestjs.com
5. **React**, Guide officiel - https://react.dev

Chapitre 1. Cadre général et méthodologie

Plan

1 Présentation de l'organisme d'accueil . . . . . ??

2 Cadre du projet 5

3 Méthodologie adaptée 10

4 Planning prévisionnel 15

Introduction

Cette mission a été réalisée dans le cadre de l'élaboration du projet de fin d'études, à l'Institut Supérieur des Études Technologiques de Radès (ISET Radès).

Durant le processus de réalisation de notre projet, l'étude de projet est une étape essentielle et obligatoire puisqu'elle assure le bon déroulement. L'étude de projet doit passer par une phase importante pour analyser l'existant afin de critiquer et d'extraire les besoins qui seront notre référence pour la fixation des objectifs du projet. Dans ce chapitre, nous allons présenter le cadre de projet, l'entreprise d'accueil, l'étude de l'existant et la solution proposée ainsi que les outils de développement et les méthodologies de conception.

1 Présentation de l'organisme d'accueil

«Solid Wall Consulting est une société de services informatiques, spécialisée dans le développement web, le développement mobile et l'intégration de solutions et d'infrastructures systèmes « Microsoft ». Forte d'une équipe dynamique et diversifiée et de profils reconnus, l'entreprise a pour objectif constant d'apporter une forte valeur ajoutée et de proposer des solutions modernes permettant aux entreprises de se démarquer sur leurs marchés respectifs. » [1] Fondée avec la vision d'accompagner les organisations publiques et privées vers l'excellence technologique et opérationnelle, Solid Wall Consulting s'est rapidement imposée comme un partenaire de confiance à l'international. » |2|.

FIGURE 1.1 – Logo SolidWall Consulting

1.1 Services de l'organisme

SolidWall Consulting propose des services conseil pour accompagner ses clients dans la réalisation de leurs projets et assure la qualité de ses prestations ainsi que le développement de solutions numériques.

L' approche agile de SolidWall Consulting garantit une gestion optimisée et par conséquent, d'un résultat de qualité.

1.2 Organigramme de l'organisme

Pour mettre en valeur la répartition des responsabilités et des fonctions au sein de l'organisation, nous présentons la structure hiérarchique de SolidWall Consulting avec ses principales divisions ainsi que les relations de subordination entre les différentes unités et postes, comme le montre la figure suivante.

2 Cadre du projet

2.1 Contexte du projet

Dans le cadre de notre projet de fin d'études à l'**Institut Supérieur des Études Technologiques de Radès**, nous avons effectué un stage du **02/02/2026** au **31/05/2026**.
Notre objectif, au sein de **SolidWall Consulting**, est de développer **« SolidMaint : une application web de gestion des contrats de maintenance »**. Cette application constitue une solution complète permettant non seulement la centralisation des contrats, mais également la garantie de la qualité de service, à travers un suivi rigoureux des interventions techniques et le respect des engagements contractuels.

2.2 Analyse de l'existant

Pour mieux comprendre les besoins, nous allons étudier les solutions existantes à deux niveaux : celles déjà en place au sein de l'entreprise SolidWall Consulting, puis celles utilisées dans le monde.

2.2.1 Etude de l'existant

L'étude de l'existant est une étape cruciale avant de commencer notre projet, parce qu'elle vise à mieux comprendre l'environnement et le critique pour respecter les exigences de l'entreprise. Il est nécessaire de clarifier la situation actuelle, d'insister sur les critiques puis de proposer des solutions.

Étude de l'existant au niveau de l'entreprise

– **Processus de gestion des contrats de maintenance :** L'entreprise utilise actuellement des documents Word pour stocker et suivre les informations relatives aux contrats. Ces documents permettent aux responsables de créer des contrats qui formalisent la relation entre l'entreprise et ses clients et qui définissent les engagements de chaque partie concernant les services de maintenance.
– **Processus de gestion des demandes clients :** le processus des demandes de maintenance repose actuellement sur des échanges par e-mail ou messagerie instantanée. Le client qui désire demander une évolution ou signaler un problème n'a que le choix de rédiger un message sans formulaire structuré, cette approche engendre une perte fréquente d'informations.

– **Processus de suivi des tickets de maintenance :** L'utilisation des fils de discussion dans des messageries (email) avec l'augmentation du nombre de clients, l'organisation atteint rapidement ses limites, l'affectation des tickets aux développeurs est réalisée manuellement par le responsable souvent via un message direct sans traçabilité.
– **Processus de la gestion des projets et des équipes :** La gestion des projets, qui inclut l'affectation des équipes et le suivi des relations avec les clients, repose sur des échanges d'e-mails pour les mises à jour de statut. Cette dispersion des informations entre plusieurs outils rend difficile pour les responsables d'avoir une vue d'ensemble en temps réel sur l'état des projets.
– **Processus de traçabilité des actions et des documents :** Les documents contractuels sont stockés de manière non structurée, soit dans des répertoires locaux, soit sur des espaces de stockage cloud, sans convention de nommage ni gestion des versions. De plus, aucun historique des modifications n'est conservé.

Étude de l'existant à l'échelle internationale

Avant de concevoir notre propre solution, nous allons étudier les solutions disponibles sur le marché afin d'analyser les fonctionnalités proposées. Cette démarche permet d'identifier les bonnes pratiques et d'orienter efficacement le développement de notre système.
Dans ce cadre, nous avons étudié plusieurs solutions existantes. Parmi celles-ci, nous nous sommes intéressés à GLPI, qui constitue un exemple représentatif dans le domaine de la gestion des demandes de maintenance :
– **GLPI (Gestion Libre de Parc Informatique)**

GLPI est une solution de gestion des services informatiques développée en 2003 et maintenue par la société française Teclib. Principalement utilisée par les équipes internes d'une organisation, selon sa configuration, elle permet de recevoir des demandes externes. Cette solution assure un système de gestion de tickets qui structure les demandes des utilisateurs. De plus, elle prend en charge différents canaux de communication, garantissant ainsi la centralisation des échanges

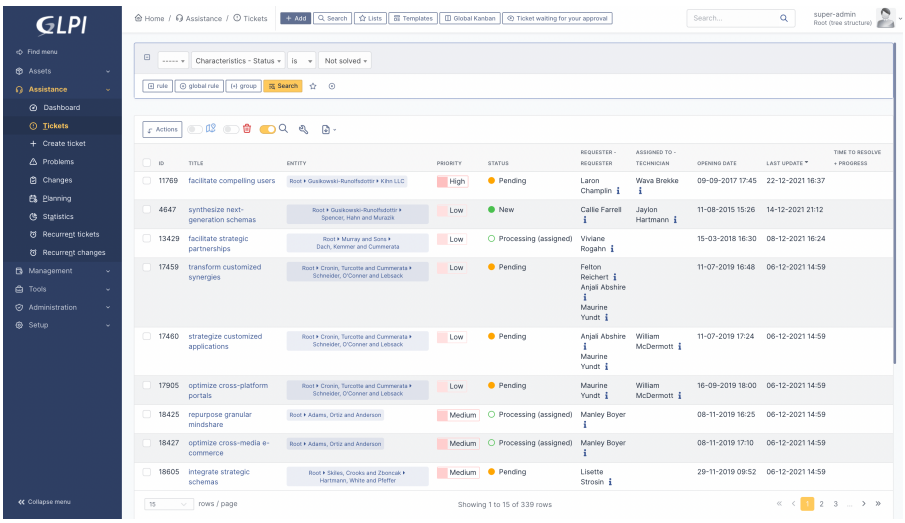

FIGURE 1.2 – Interface de GLPI

Par la suite, nous nous sommes intéressés à la solution Odoo, qui couvre plusieurs besoins de gestion d'entreprise.

– **Odoo**

Odoo est une solution de gestion d'entreprise développée en 2005 basée en Belgique. elle couvre différents besoins métiers des entreprises, notamment à travers un module Helpdesk qui assure la gestion des demandes de maintenance sous forme de tickets organisés et suivis selon différents statuts. Elle peut créer les tickets via plusieurs canaux tels que l'email ou les formulaires en ligne.
La solution intègre également des fonctionnalités d'automatisation, notamment l'attribution automatique des tickets et l'envoi d'alertes.

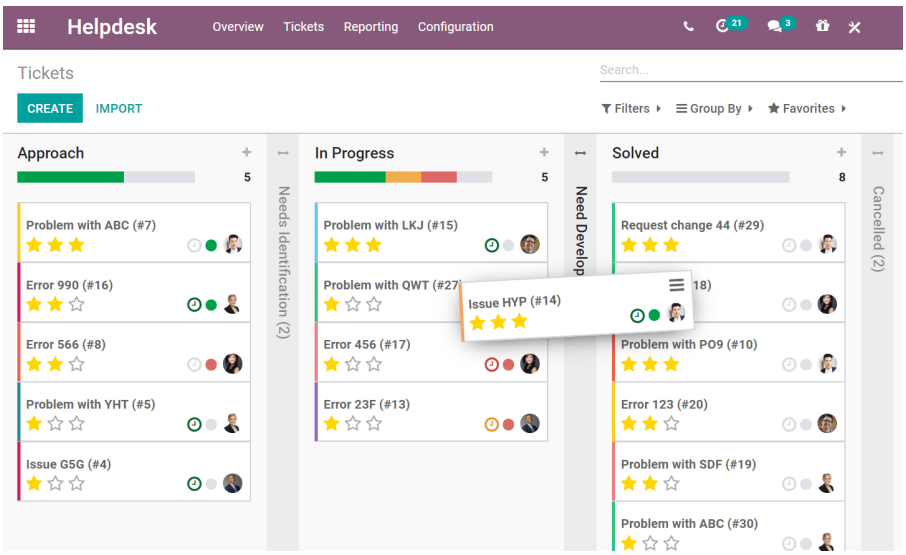

FIGURE 1.3 – Interface de Odoo

2.2.2 Critique de l'existant

Actuellement, la gestion des contrats de maintenance au sein de la direction de la gestion de SolidMaint se fait à partir d'Excel et par communication via mail. Cette approche présente des failles.

Limitations du processus actuel

- **processus fragmenté :** l'utilisation séparée d'Excel et la communication via e-mail fragmentent la gestion. Les informations dispersées compliquent la gestion des composants manquants. Sans lien structuré entre eux, il devient alors difficile de retrouver l'historique complet d'une intervention.
- **Absence de traçabilité :** L'absence de système de traçabilité rend le suivi des interventions difficile. Cela peut entrainer une perte de visibilité sur l'état en cas de conflit avec un client ou lors d'un audit interne. Ce qui représente un risque pour la crédibilité et la fiabilité de la société.
- **Absence de sécurité :** Utiliser des outils généralistes pour gérer des données sensibles expose l'entreprise à des risques de sécurité. Ces plateformes ne garantissent pas les niveaux de sécurité nécessaires pour protéger efficacement les informations confidentielles.
- **Risque de perte d'informations :** En raison de la fragmentation de processus, il existe un risque accru de perte d'informations importantes. Les documents considérable sont stockés de manière non structurée, ce qui peut être difficile à suivre et à consolider.
- **Complexité de la communication :** la communication entre l'équipe et avec les clients repose sur des échanges dispersés (emails, messages), ce qui rend le flux des demandes et de configuration flou et entraîne un retard dans le traitement des interventions.

Limitations des solutions existantes

- **Complexité d'utilisation :** Les solutions GLPI et Odoo proposent plusieurs modules, ce qui peut rendre la gestion des demandes plus complexe. Les utilisateurs doivent disposer d'une certaine expérience, ce qui peut ralentir leur adoption au sein de l'organisation.
- **Coût des licences :** La majorité des services sont payants ou nécessitent un abonnement pour accéder à des fonctionnalités avancées, ce qui représente un coût élevé pour l'organisation.
- **Dépendance aux extensions :** De nombreuses fonctionnalités avancées nécessitent des extensions, ce qui peut engendrer des problèmes de compatibilité lors des mises à jour.

2.2.3 Solution proposée

Dans le contexte des défis exposés, nous proposons la mise en place de SolidMaint, un outil web pour rationaliser le bon déroulement et l'accélération du processus. SolidMaint favorisera une traçabilité mémorable des tâches réalisées tout en offrant une représentation claire des données. En effet, notre application SolidMaint comprend quatre modules principaux :

Le premier module, adressé à la **Gestion des Utilisateurs**, permettra aux responsables de superviser et de gérer de manière précise les profils des utilisateurs. Ils auront la capacité d'attribuer des connexions spécifiques et de distribuer les rôles avec des conditions rigides.
De plus, ce modèle offrira la possibilité de gérer les rôles, les permissions. Ce module permet à chaque utilisateur connecté de consulter et de modifier son propre profil.

En ce qui concerne le deuxième module, centré sur la **Gestion des engagements**, ce module englobe la Gestion des contrats qui permet au responsable de formaliser les engagements contractuels liant SolidWal Consulting à ses clients. De plus, le responsable configure l'accord du niveau de service (SLA), cette fonctionnalité garantit la qualité fondamentale, liée à la satisfaction client et à la crédibilité de la société. Ce module met l'accent sur la Gestion des Projets, le responsable a la possibilité de définir les clients, de détailler les projets. De plus, la fonctionnalité d'affectation des membres de l'équipe à chaque projet garantit ainsi une utilisation efficace des compétences disponibles.

Dans le cadre du troisième module, centré sur la **Gestion des Demandes et des Tickets**,
l'objectif principal est de structurer le cycle de vie complet d'une demande client de
maintenance depuis sa soumission jusqu'à sa clôture en passant par la validation. Dès qu'une demande est validée, SolidMaint génère automatiquement des tickets, les responsables auront accès pour affecter chaque ticket au membre de l'équipe.

Enfin, le quatrième module, consacré à la **Gestion des événements**, offrira aux utilisateurs la possibilité de recevoir des informations en temps réel. En intégrant un moteur d'alertes SLA assurant le respect des engagements contractuels, ainsi que la surveillance des dates d'expiration des contrats de maintenance en générant des alertes préventives. De plus, ce module offrira l'optimisation de la prise de décision, garantissant ainsi la réussite globale des activités de l'entreprise.

3 Méthodologie adaptée

La méthodologie adoptée est une étape importante car elle garantit la compréhension du projet pour trouver un meilleur résultat qui satisfait les besoins des clients.

Ainsi, il est important de reposer sur un langage de modélisation unifié pour formaliser les solutions de manière claire. D'aboutir à une représentation abstraite et simplifiée du projet

3.0.1 Méthodologie traditionnelle

L'approche traditionnelle de la gestion de projet s'articule autour d'une série d'étapes consécutives, où l'approbation de la phase précédente est essentielle pour progresser vers la suivante. Cette méthode suppose que les projets gérés de manière traditionnelle se déroulent en phases distinctes et structurées. Cette logique est représentée par divers modèles de cycle de vie, incluant le cycle en cascade, le cycle en V et le cycle en spirale.

3.0.2 Méthodologie Agile

« La méthodologie Agile décompose le travail en phases, mettant l'accent sur la livraison continue et l'amélioration. Agile profite aux équipes en permettant une planification adaptative, une exécution rapide et une évaluation continue, garantissant des résultats plus réactifs et plus réussis. » |3|
La méthodologie Agile est adoptée pour pouvoir répondre rapidement aux commentaires des clients, valorise la collaboration plus que la négociation contractuelle en proposant une version minimale puis intègre des fonctionnalités supplémentaires. La méthode agile est une méthode moderne qui offre la souplesse et la performance à la gestion de projet.

La méthodologie Agile est adoptée pour répondre rapidement aux retours des clients. Elle valorise la collaboration plutôt que la négociation contractuelle, en proposant d'abord une version minimale, puis en intégrant des fonctionnalités supplémentaires. La méthode Agile est une démarche moderne qui offre souplesse et performance dans la gestion de projet.
Afin d'assurer une meilleure qualité et l'efficacité de la solution, nous avons choisi **les méthodologies agiles**. qui portent sur la forte collaboration des clients dès le début.

3.0.3 Les douze principes de l'Agile

Le tableau ci-dessous présente les douze principes Agile qui définissent les valeurs clés pour le développement des logiciels.

N°	Les 12 principes du Manifeste Agile
1	Prioriser la satisfaction du client à travers une livraison rapide.
2	Accepter les changements même tard dans le développement.
3	Livrer fréquemment des versions opérationnelles de l'application.
4	Assurer quotidiennement une coopération entre l'équipe du projet et les gens des métiers.
5	Construire des projets autour d'individus motivés, leur donnant le soutien nécessaire.
6	Favoriser le dialogue direct.
7	Mesurer l'avancement du projet en fonction de l'opérationnalité du produit
8	Adopter un rythme constant et soutenable par tous les intervenants du projet.
9	Contrôler continuellement l'excellence de la conception et la bonne qualité technique.
10	Privilégier la simplicité en évitant le travail inutile.
11	Auto organiser et responsabiliser les équipes.
12	Améliorer régulièrement l'efficacité de l'équipe en ajustant son comportement.

TABLE 1.1 – Les 12 principes du développement Agile.

3.0.4 Les valeurs agiles

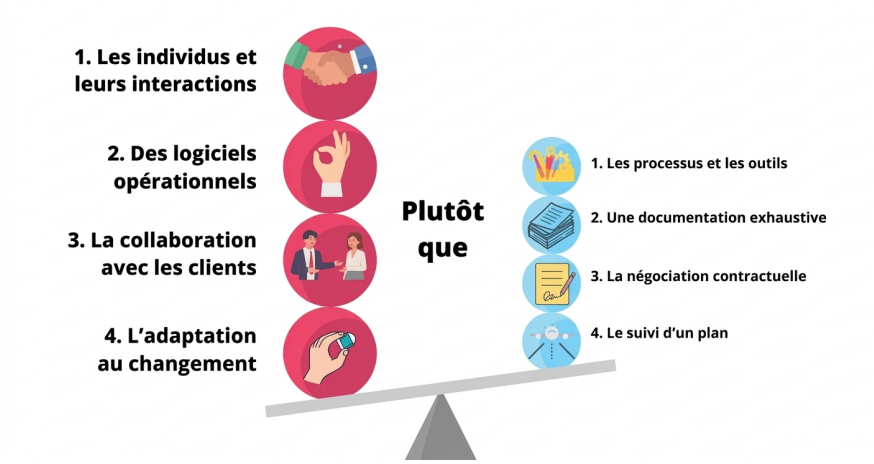

FIGURE 1.4 – les valeurs agiles

Cette image marque les quatre valeurs importantes de la méthode agile qui sont des principes clés dans la gestion des projets, constituent un gain en productivité. Les valeurs Agile sous-tendent l'accent sur :

- Les individus et leurs interactions plus que les processus et les outils
- Des logiciels opérationnels plus qu'une documentation exhaustive
- La collaboration avec les clients plus que la négociation contractuelle

– L'adaptation au changement plus que le suivi d'un plan

Ces valeurs renforcent la prise de décision et favorisent la réactivité dans le travail de l'équipe.

3.1 Les principales méthodes Agiles

Le tableau suivant dévoile les différentes méthodes agiles les plus populaires et quelques critères de comparaison.

Critères	Scrum	Kanban	Extreme Programming (XP)
Type de projet	Incrémental	Continu	Technique
Critère d'évaluation	Vélocité	Fluidité	Qualité
Gestion des risques	Détection	Visualisation	Prévention
Implication client	Ponctuelle	Variable	Très forte et continue

TABLE 1.2 – Les principales méthodes Agiles.

3.1.1 Choix de la méthode de travail

Nous avons choisi la méthodologie **SCRUM** comme approche pour la réalisation de notre projet, notre choix est fondé sur de nombreux avantages qui répondent à nos besoins, notamment :

– le travail collaboratif fortement impliqué.
– l'adaptation au changement grâce au cycle itératif.
– la planification et l'exécution des projets prévisibles et efficaces.
– la communication régulière et en toute transparence qui permet à l'équipe d'avancer.
– la détection rapide des erreurs.

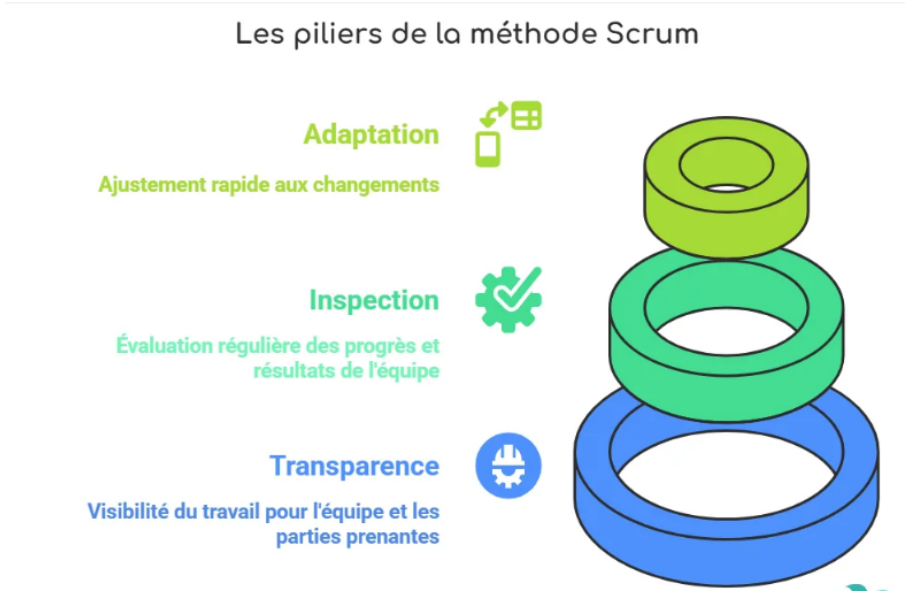

FIGURE 1.5 – les piliers de la méthodologie

3.2 Fondements de SCRUM

Scrum est une méthode agile. Il s'appuie sur le découpage des projets en sprints, qui sont des cycles de développement très courts. Les fonctionnalités sont présentées sous forme de user stories qui décrivent quelles fonctionnalités sont nécessaires et à quel point elles sont importantes.Le principe de Scrum est de maximiser la productivité et d'assurer en même temps la satisfaction du client. Scrum repose aussi sur trois grands principes : la transparence, l'inspection et l'adaptation.

3.2.1 Pertinence de SCRUM

Le choix de Scrum comme méthodologie pour la réalisation de notre projet est fondé sur plusieurs raisons. En effet, cette méthode propose une approche temporelle et itérative du développement qui facilite la livraison régulière et le suivi de l'avancement à chaque étape. Elle est spécialement adaptée à l'évolution des besoins en assurant la flexibilité.

3.2.2 Rôles de SCRUM

L'équipe Scrum est auto-organisée et prend les décisions sur la manière dont le travail sera réalisé pour atteindre l'objectif du sprint, elle est composée d'un propriétaire de produit, d'un Scrum Master et d'une équipe de développement :
Le **Product Owner**, c'est l'expert du domaine métier du projet qui porte la vision du produit à réaliser. Ce rôle repose sur la représentation de la voix du client et veille à ce que le produit reste aligné sur la vision stratégique de l'organisation.
Le **Scrum Master**, Son objectif est d'assurer la productivité de l'équipe dans l'application de Scrum. Sa collaboration avec l'équipe vise à maximiser la performance globale, en alignant les efforts sur les objectifs stratégiques.

L' **Équipe de développement**, ce sont les personnes chargées de la réalisation du sprint, ayant pour mission de transformer les besoins exprimés en fonctionnalités utilisables du produit.

3.2.3 Evènements Scrum

- **Sprint :** Un sprint est une itération de développement d'une durée fixe (généralement 2 à 4 semaines) durant laquelle une version potentiellement livrable du produit est réalisée.
- **Sprint Planning :** Réunion de planification du sprint où l'équipe définit les objectifs et sélectionne les éléments du Product Backlog à réaliser pendant le sprint.
- **Daily Scrum :** Réunion quotidienne de 15 minutes permettant à l'équipe de synchroniser ses actions, d'identifier les obstacles et d'ajuster le plan de travail.
- **Revue de Sprint :** Réunion à la fin du sprint pour présenter le travail accompli, recueillir les retours et adapter le Product Backlog si nécessaire.
- **Sprint Rétrospective :** Réunion d'amélioration continue où l'équipe analyse le déroulement du sprint, identifie ce qui a bien fonctionné et ce qui doit être amélioré.

3.2.4 Artéfacts Scrum

- **Product Backlog :** Liste priorisée de toutes les fonctionnalités, évolutions, corrections et tâches à réaliser pour le produit.
- **Sprint Backlog :** Sous-ensemble du Product Backlog sélectionné pour le sprint en cours, accompagné d'un plan pour livrer l'incrément.
- **Incrément :** Somme de tous les éléments du Product Backlog terminés durant un sprint et les sprints précédents, représentant une version utilisable du produit.

3.2.5 Cycle de vie

Le cycle de vie Scrum se compose de sprints successifs, chacun comprenant une planification, le développement, des réunions quotidiennes, une revue et une rétrospective. Ce processus itératif permet d'adapter le produit en continu aux besoins du client et d'améliorer l'équipe à chaque itération.

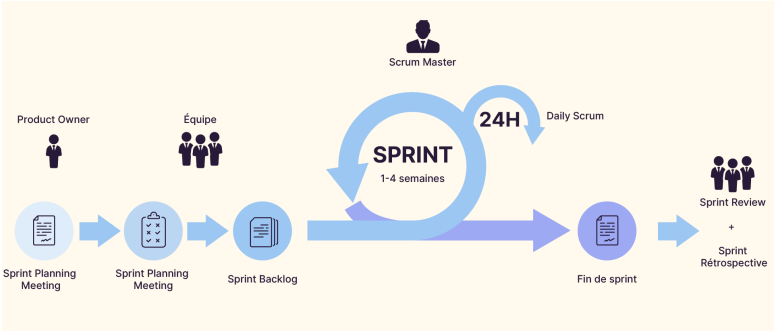

FIGURE 1.6 – Cycle de vie Scrum

3.3 Langage de modélisation UML – Unified Modeling Language

« Le langage de modélisation unifié (UML) est un langage permettant de spécifier, visualiser, construire et documenter les artefacts des systèmes logiciels, ainsi que de modéliser des systèmes métier et d'autres systèmes non logiciels. L'UML représente un ensemble de bonnes pratiques d'ingénierie qui ont fait leurs preuves dans la modélisation de systèmes vastes et complexes. » |4|

FIGURE 1.7 – Logo UML

Pour concevoir notre application, nous avons choisi UML comme langage de modélisation et notre choix s'appuie sur plusieurs points forts tels que la communication entre l'équipe et la cohérence architecturale qui se fait de manière simplifiée et standardisée.

4 Planning prévisionnel (Diagramme de Gantt)

« Un diagramme de Gantt est un outil d'organisation et de gestion de projets. Il s'agit d'un diagramme à barres horizontales qui présente toutes les tâches d'un projet en fonction d'un calendrier. Il aide les chefs de projet à visualiser les tâches requises et à suivre l'avancement de leur projet. » |5| Dans le cadre du développement de notre projet SolidMaint, le planning prévisionnel a été réalisé pour organiser les différentes phases de développement. ci-dessus illustre cette planification :

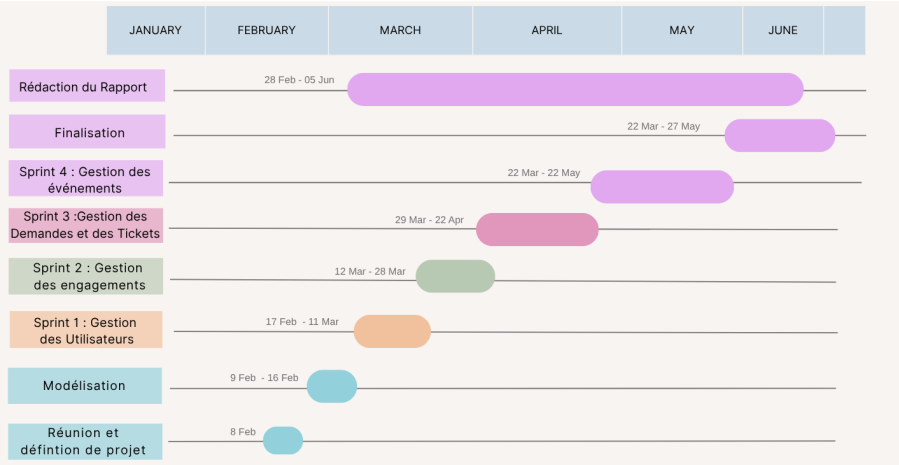

FIGURE 1.8 – Diagramme de Gantt

Références

[1] SolidWall Consulting. Site officiel. Disponible sur : https://solidwall.com.tn/ (Consulté le 21/04/2026) [2] SolidWall Consulting. Publication LinkedIn PFE 2025-2026. Disponible sur : https://www.linkedin.com/posts/solid-wall-consulting_pfe-book-2025-2026-activity-7402287392465862656-MR2f (Consulté le 21/04/2026) [3] Atlassian. Qu'est-ce que la méthodologie Agile ? Disponible sur : https://www.atlassian.com/fr/agile (Consulté le 21/04/2026) [4] OMG. Why is UML important ? Disponible sur : https://www.omg.org/uml/why-uml-is-important.htm (Consulté le 21/04/2026) [5] Miro. Qu'est-ce qu'un diagramme de Gantt ? Disponible sur : https://miro.com/fr/planification-strategique/qu-est-ce-qu-un-diagramme-de-gantt/ (Consulté le 21/04/2026) [6] Openfire. (s.d.). Contrat de maintenance : comment simplifier la gestion de vos contrats d'entretien annuels ? Disponible sur : http://openfire.fr/blog/toutes-actualites-3/contrat-de-maintenance-comment-simplifier-la-gestion-de-vos-contrats-dentretien-annuels-74 (Consulté le 21/04/2026)

Chapitre 2. Analyse et spécifications des besoins

Plan

1 Spécification des besoins 18

2 Modélisation des besoins 22

3 Pilotage du projet avec Scrum 22

4 Environnement de développement 39

5 Architecture de l'application . . . . . ??

Introduction

1 Spécification des besoins

L'étape de spécification de besoin est importante pour prouver qu'un besoin réel existe, elle permet de définir explicitement les objectifs à atteindre.
Pour cela nous allons identifier les acteurs, leurs besoins fonctionnels et les besoins non fonctionnels.

1.1 Identification des acteurs

Un acteur représente une personne, une organisation ou un autre système qui interagit avec le système.
Les acteurs définis dans notre projet sont :
Administrateur : Responsable de gérer les utilisateurs, les rôles et permissions, les clients, des projets ainsi que des contrats
Responsable de projet : Supervise les projets qui lui sont assignés, approuve les demandes de maintenance soumises, assigne les développeurs aux projets, suit les tickets et les assigne aux développeurs, et assure le suivi des SLA.
Développeur : Assigné pour gérer les tickets, consulter les documents liés aux tickets
Client : interagir pour créer des demandes de maintenance et consulter ses propres demandes et leur état.

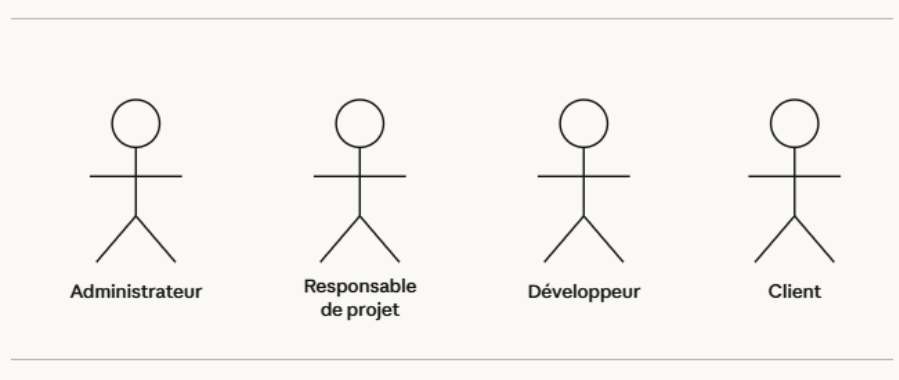

FIGURE 2.1 – Représentation des acteurs en UML

1.2 Identification des besoins fonctionnels

Une définition précise des besoins fonctionnels permet d'établir un plan de projet solide. Il s'agit de spécifications qui détaillent les opérations que le service doit être capable d'exécuter.
Notre application a pour objectif de répondre à chaque besoin de chaque utilisateur pour faciliter la gestion des projets et des incidents.

1.2.1 Besoins fonctionnels de l'acteur « Client »

- Consulter et gérer ces demandes :
  - **Soumettre une demande :** exprimer une demande de maintenance afin d'informer le responsable de projet.
  - **Consulter ses demandes :** accéder à la liste de ses demandes soumise avec leur statut d'avancement
- Consulter ces Tickets :
  - **suivre l'avancement des tickets :** consulter l'état de ses tickets afin de vérifier si sa demande est résolue

1.3 Identification des besoins non fonctionnels

durant la réalisation de ce projet, on prend en considération les besoins non fonctionnels qui désignent les attributs de qualité d'un système
Les critères que notre plateforme doit satisfaire sont :

- **Ergonomie :**

  Nous penserons à minimiser la charge cognitive des utilisateurs et à maximiser leur efficacité opérationnelle. Nous intégrerons la gestion dynamique des thèmes (mode clair / mode sombre). Cette fonctionnalité assurera un confort visuel optimal dans toutes les conditions de travail.
  Nous implémenterons une architecture responsive ; l'interface s'adaptera automatiquement à la taille des écrans (desktop et téléphone) afin d'offrir une expérience fluide.
  Nous nous reposerons sur un tableau de bord centralisé qui assurera la logique de progression, afin de garantir que l'utilisateur comprenne toujours l'état du système.
  Nous renforcerons les mises à jour instantanées via des composants réactifs ; cela éliminera les interruptions des rechargements de page classiques.
  Nous soulignerons la simplicité d'utilisation de l'application, ainsi que la cohérence du design et des polices, offrant une impression professionnelle.

- **Sécurité :**

  la sécurité sera une priorité absolue pour notre application. Nous intégrerons des mesures robustes telles que le hachage des mots de passe et l'utilisation de tokens à l'aide de la bibliothèque **bcrypt**.
  Nous implémenterons un mécanisme de contrôle d'accès reposant sur le principe de la restriction des accès selon les droits attribués **(RBAC)**. Ces pratiques renforceront la protection des données sensibles de nos utilisateurs.
  Afin de garantir une authentification sécurisée, l'application prendra en charge la connexion via Google et GitHub, tout en respectant la sécurité optimale d'authenticité. Si l'utilisateur ne dispose pas des droits nécessaires pour se connecter, une demande d'inscription est créée en base de données et soumise à validation par un administrateur, l'utilisateur restera bloqué tant que sa demande ne sera pas approuvée.
  Par ailleurs, la génération d'un token de réinitialisation de mot de passe sera associée à une durée de validité d'une heure et à un usage unique. Afin d'assurer un audit complet, notre application garantira la journalisation de toutes les actions avec les informations nécessaires pour identifier l'acteur ainsi que l'action réalisée.
  Lors de la création d'un contrat avec l'option de l'upload, chaque fichier téléversé sera vérifié à l'aide d'un hash, permettant de détecter les doublons et d'éviter les uploads multiples d'un même contrat, garantissant ainsi l'unicité contractuelle.
  Enfin, lors de la création d'un utilisateur, le système générera automatiquement un mot de passe. Par conséquent, lors de sa première connexion, l'utilisateur sera contraint de le modifier afin de renforcer la sécurité de son compte.

- **Disponibilité** :

  Nous serons résolus à faire de la disponibilité et de la fiabilité des critères importants dans notre plateforme.

  Afin de maintenir une surveillance continue des délais de réponse et de résolution des tickets en relation avec les engagements de service du contrat, nous insisterons sur la garantie d'une communication en temps réel à l'aide d'une messagerie entre la société et ses clients ainsi qu'entre l'équipe.

  Nous mettrons en place des mécanismes de persistance des informations afin de garantir qu'elles ne se perdent pas. Pour cela, nous utiliserons une diffusion instantanée via le protocole de communication **WebSocket**, ainsi qu'un mécanisme de persistance en cas de déconnexion ou d'indisponibilité temporaire du réseau. Les notifications seront enregistrées en base de données avec une synchronisation automatique permettant de livrer les messages en attente.

  Afin d'éviter toute expiration d'un contrat, nous intégrerons un mécanisme d'alerte sur plusieurs durées (les 30 derniers jours, les 15 derniers jours et les 7 derniers jours). Ce système garantira que les responsables soient informés en temps opportun.

2 Modélisation des besoins

L'analyse des besoins fonctionnels se poursuit par une modélisation dynamique à l'aide du diagramme de cas d'utilisation UML. Ce diagramme offre une vision synthétique des fonctionnalités offertes par le système du point de vue de chaque acteur, sans se préoccuper de leur ordre d'exécution ou des détails techniques.

La figure 2.2 présente le diagramme de cas d'utilisation global de notre plateforme de service desk. Ce diagramme structure les interactions entre les quatre acteurs principaux identifiés et le système. Chaque acteur possède un périmètre d'actions bien défini.

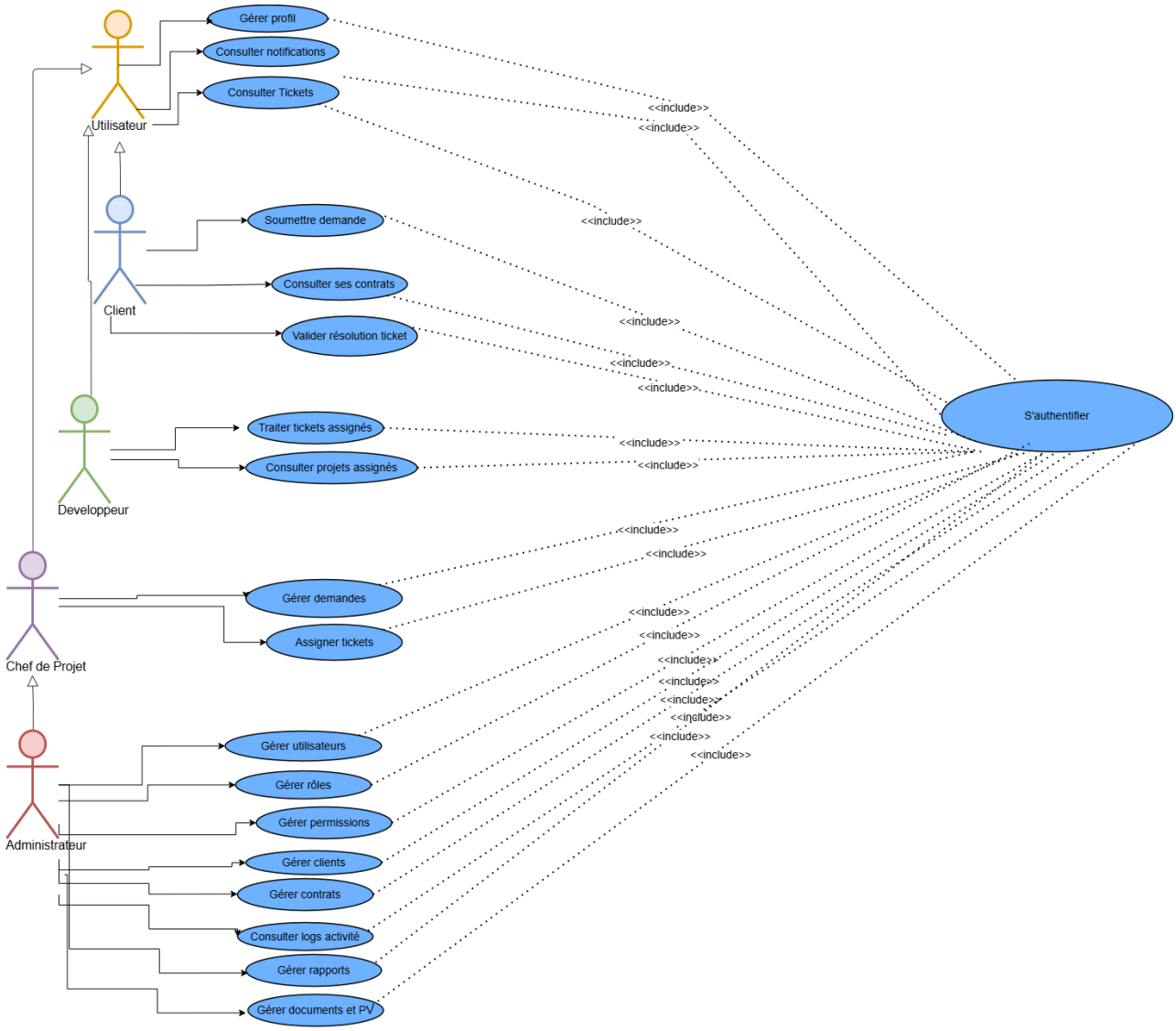

FIGURE 2.2 – Diagramme de cas d'utilisation global du système

3 Pilotage du projet avec Scrum

3.1 Rôles de SCRUM

Dans notre projet, Monsieur Mejdi Dhokar sera le Product Owner, Madame Maissa Ben Fredj sera le Scrum Master, et nous, Rihab Chourou et Eya Gharbi, formons l'équipe de développement.

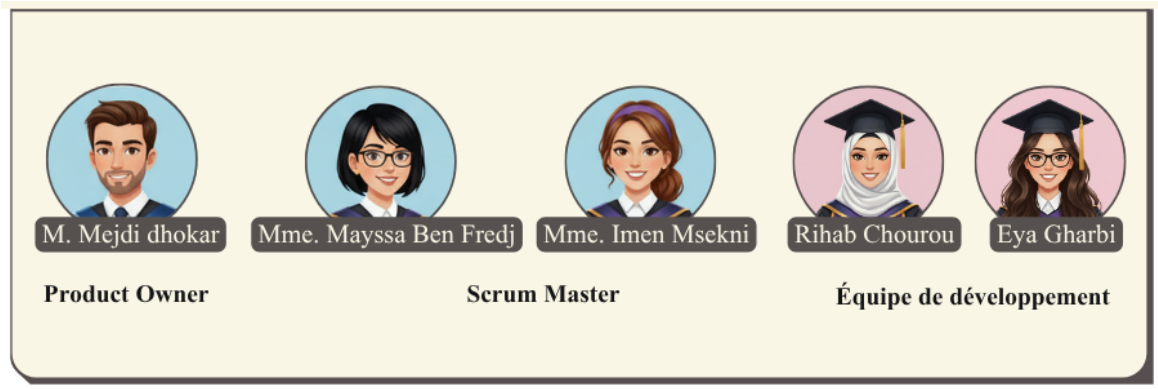

FIGURE 2.3 – Équipe scrum et rôles

3.1.1 Backlog de produit

Comme évoqué dans le premier chapitre, le Product Backlog constitue l'un des artefacts fondamentaux de la méthodologie Scrum. Il permet de formaliser les exigences fonctionnelles ainsi que les fonctionnalités attendues du système. Le tableau ci-dessous présente le Product Backlog de notre solution SolidMaint, structuré selon les éléments suivants :

- **Id EPIC** : Identifiant unique attribué à chaque EPIC afin de faciliter son identification et sa gestion dans le système de projet.
- **EPIC** : Ensemble de tâches interconnectées visant à atteindre un objectif majeur dans la gestion agile de projet ou le développement logiciel.
- **Id User Story** : Valeur unique et incrémentale associée à chaque user story pour assurer sa traçabilité.
- **User Story** : Description de la fonctionnalité du point de vue de l'utilisateur final, suivant le format "En tant que [rôle], je souhaite [action] afin de [bénéfice]".
- **Critères d'acceptation** : Conditions spécifiques et mesurables qui doivent être remplies pour que la user story soit considérée comme terminée et acceptée par le Product Owner.
- **Priorité** : Niveau d'importance défini par le Product Owner pour chaque tâche, permettant de prioriser les développements.

Légende des priorités : + : Faible, ++ : Moyenne, +++ : Élevée.

TABLE 2.1 – Product Backlog

ID Epic	Épic	ID US	User Story	Critères d'acceptation	Priorité
1	Authentification et Sécurité	US01	En tant qu'utilisateur, je souhaite m'authentifier avec email et mot de passe afin d'accéder à mes informations de manière sécurisée.	L'utilisateur authentifié avec succès accède à son tableau de bord personnalisé selon son rôle.	+++
		US02	En tant qu'utilisateur, je souhaite réinitialiser mon mot de passe afin de retrouver l'accès à mon compte.	L'utilisateur reçoit un email avec un lien valide 24h et peut définir un nouveau mot de passe conforme aux règles de sécurité.	+++
		US03	En tant qu'utilisateur, je souhaite m'authentifier via Google ou GitHub afin de me connecter rapidement sans créer de compte.	L'utilisateur peut se connecter via son compte Google ou GitHub et accéder immédiatement à l'application.	++
		US04	En tant qu'utilisateur, je souhaite me déconnecter de la plateforme afin de sécuriser ma session.	Après déconnexion, l'utilisateur ne peut plus accéder aux pages protégées sans se reconnecter.	++
		US05	En tant qu'utilisateur, je souhaite que mon compte soit protégé contre les tentatives de connexion malveillantes afin de garantir la sécurité de mes données.	Après 3 tentatives échouées, le compte est temporairement verrouillé et l'utilisateur reçoit une notification par email.	+++
2	Gestion des utilisateurs et profils	US06	En tant qu'utilisateur, je souhaite gérer mon profil afin que mes informations soient toujours à jour.	L'utilisateur peut modifier et sauvegarder ses informations personnelles qui sont validées avant enregistrement.	++
		US07	En tant qu'administrateur, je souhaite créer des utilisateurs afin de gérer les accès au système.	Le nouvel utilisateur reçoit un email d'invitation avec un mot de passe temporaire pour sa première connexion.	+++

		US08	En tant qu'administrateur, je souhaite modifier les informations d'un utilisateur afin de maintenir des données à jour.	Les modifications sont enregistrées avec traçabilité et l'utilisateur concerné est notifié des changements.	++
		US09	En tant qu'administrateur, je souhaite activer et désactiver un compte utilisateur afin de contrôler l'accès au système.	Un compte désactivé ne peut plus se connecter et l'utilisateur reçoit une notification du changement de statut.	+++
		US10	En tant qu'administrateur, je souhaite consulter la liste des utilisateurs afin de visualiser et gérer efficacement les comptes.	La liste complète des utilisateurs est affichée avec possibilité de filtrer, rechercher et exporter les données.	++
		US11	En tant qu'utilisateur, je souhaite consulter l'historique de mes activités afin de suivre mes actions dans le système.	L'utilisateur visualise toutes ses actions passées avec possibilité de filtrer par date et type d'action.	++
3	Gestion des rôles et permissions	US12	En tant qu'administrateur, je souhaite gérer des rôles afin de structurer les accès au système.	L'administrateur peut créer, modifier et archiver des rôles avec vérification des utilisateurs associés.	+++
		US13	En tant qu'administrateur, je souhaite attribuer des permissions granulaires aux rôles afin de contrôler précisément les accès.	L'administrateur sélectionne les permissions par catégorie et visualise un aperçu avant validation.	+++
		US14	En tant qu'administrateur, je souhaite consulter la matrice rôles-permissions afin de visualiser rapidement les droits de chaque rôle.	La matrice affiche clairement tous les rôles et leurs permissions avec possibilité de filtrer et exporter.	++
4	Gestion des demandes d'inscription OAuth	US15	En tant qu'utilisateur externe, je souhaite demander l'accès via OAuth afin de m'inscrire facilement.	La demande est enregistrée et l'administrateur reçoit une notification pour traitement.	++

		US16	En tant qu'administrateur, je souhaite valider ou rejeter les demandes d'inscription OAuth afin de contrôler les accès.	L'utilisateur reçoit un email de confirmation ou rejet et peut accéder à l'application si validé.	+++
5	Gestion des clients	US17	En tant qu'administrateur, je souhaite créer un profil client afin de centraliser ses informations.	Le profil client est créé avec toutes les informations nécessaires et un identifiant unique est généré.	+++
		US18	En tant qu'administrateur, je souhaite consulter la liste des clients afin de visualiser les clients existants.	La liste complète des clients est accessible avec possibilité de filtrer, rechercher et exporter.	++
		US19	En tant qu'administrateur, je souhaite consulter le profil détaillé d'un client afin de visualiser toutes ses informations.	Toutes les informations du client sont affichées incluant contrats, projets et historique des interactions.	++
		US20	En tant qu'administrateur, je souhaite modifier les informations d'un client afin de maintenir des données à jour.	Les modifications sont enregistrées avec traçabilité et le client est notifié si ses coordonnées changent.	++
		US21	En tant qu'administrateur, je souhaite activer et désactiver un compte client afin de contrôler l'accès.	Un compte client désactivé ne peut plus accéder au système et reçoit une notification du changement.	+++
		US22	En tant qu'administrateur, je souhaite ajouter des notes privées sur un client afin de conserver des informations internes.	Les notes sont enregistrées avec date et auteur, visibles uniquement par l'équipe interne.	+
		US23	En tant qu'administrateur, je souhaite imprimer l'annuaire d'un client afin de disposer d'une version papier.	Un document PDF professionnel contenant toutes les informations du client est généré.	+
6	Gestion des projets et équipes	US24	En tant qu'administrateur, je souhaite créer un projet rattaché à un client afin d'organiser les interventions.	Le projet est créé et rattaché au client avec un chef de projet assigné.	+++

		US25	En tant qu'administrateur, je souhaite consulter la liste des projets afin de piloter l'activité.	La liste complète des projets est affichée avec possibilité de filtrer par client, statut ou chef de projet.	++
		US26	En tant qu'administrateur, je souhaite affecter des développeurs à un projet afin de constituer l'équipe.	Les développeurs sont affectés au projet et peuvent visualiser les informations du projet.	+++
		US27	En tant que membre d'équipe, je souhaite consulter les informations du projet afin de comprendre le contexte.	Toutes les informations du projet sont accessibles incluant client, objectifs, équipe et contrat.	++
7	Gestion des contrats de maintenance	US28	En tant qu'administrateur, je souhaite créer un contrat de maintenance afin de formaliser les engagements de service avec le client.	Le contrat est créé avec client, dates, type SLA, cycle facturation et configuration des alertes d'expiration.	+++
		US29	En tant qu'administrateur, je souhaite consulter la liste des contrats afin de suivre les engagements contractuels.	La liste affiche tous les contrats avec filtres, indicateurs visuels d'expiration et possibilité d'export.	++
		US30	En tant qu'administrateur, je souhaite recevoir des alertes avant l'expiration des contrats afin d'anticiper les renouvellements.	Des alertes automatiques sont envoyées à J-30, J-15 et J-7 par email, in-app et Slack.	+++
		US31	En tant qu'administrateur, je souhaite archiver un contrat afin de conserver l'historique.	Le contrat est archivé avec un changement de statut, avec possibilité de restauration ultérieure.	++
		US32	En tant que client, je souhaite demander la résiliation de mon contrat afin d'interrompre le service.	La demande de résiliation est créée avec motif et date souhaitée, l'administrateur reçoit une notification et peut traiter la demande.	+++
		US33	En tant qu'administrateur, je souhaite résilier un contrat afin de l'interrompre avant terme.	Le contrat est résilié avec date et motif, les tickets ouverts sont clôturés et le client est notifié.	+++

		US34	En tant qu'administrateur, je souhaite renouveler un contrat afin de prolonger la relation.	Un nouveau contrat est créé à partir de l'existant avec lien entre les deux et notification au client.	+++
		US35	En tant que chef de projet, je souhaite consulter les contrats dans un calendrier afin de visualiser les dates d'expiration.	La vue calendrier mensuel/annuel affiche les contrats avec code couleur selon l'échéance et export iCal.	++
		US36	En tant qu'administrateur, je souhaite centraliser les documents d'un contrat afin de les retrouver facilement.	Les documents sont uploadés avec versioning automatique et associés au contrat.	+
		US37	En tant qu'administrateur, je souhaite configurer les règles SLA globales afin de définir les standards.	Les délais de réponse et résolution sont configurés par priorité (MINEUR, MAJEUR, CRITIQUE).	+++
8	Gestion des demandes clients	US38	En tant que client, je souhaite soumettre une demande de maintenance afin de signaler un besoin.	La demande est créée avec pièces jointes (max 5), un numéro unique est généré et une confirmation est envoyée.	+++
		US39	En tant que chef de projet, je souhaite consulter les demandes en attente afin de les traiter.	La liste filtrée affiche les demandes EN_ATTENTE triées par date avec indicateurs de priorité.	+++
		US40	En tant que chef de projet, je souhaite valider une demande afin de la convertir en ticket.	La demande est validée avec définition de catégorie et priorité, un ticket lié est créé automatiquement.	+++
		US41	En tant que chef de projet, je souhaite demander des informations complémentaires afin de clarifier une demande.	La demande passe en statut INFOS_REQUISES avec questions et relance automatique après 48h.	++
		US42	En tant que client, je souhaite compléter ma demande afin de fournir les informations manquantes.	Le client répond au formulaire et la demande retourne automatiquement au statut EN_ATTENTE.	++

		US43	En tant que chef de projet, je souhaite requalifier une demande afin de corriger sa catégorie ou sa priorité.	Le type et la priorité sont modifiés avec justification obligatoire et traçabilité.	++
		US44	En tant qu'utilisateur, je souhaite commenter une demande afin de communiquer sur le contexte.	Les commentaires sont ajoutés avec éditeur riche, mentions @utilisateur et pièces jointes possibles.	++
		US45	En tant qu'utilisateur, je souhaite consulter l'historique d'une demande afin de suivre son évolution.	La timeline affiche tous les événements (création, validation, commentaires, conversion) avec filtres.	++
9	Gestion des tickets	US46	En tant que développeur, je souhaite visualiser mes tickets dans un tableau Kanban afin de gérer mes tâches.	Colonnes OUVERT, EN_COURS, EN_REVUE, RESOLU, FERME. Drag & drop et indicateurs SLA.	+++
		US47	En tant que chef de projet, je souhaite assigner un ticket à un développeur afin de répartir la charge.	Sélection du développeur avec vérification disponibilité/compétences, alerte surcharge.	+++
		US48	En tant que chef de projet, je souhaite assigner un ticket à plusieurs développeurs afin de collaborer sur des tâches complexes.	Sélection multiple, rôles principal/support, répartition du temps estimé.	++
		US49	En tant que développeur, je souhaite me désassigner d'un ticket afin de libérer ma charge.	Désassignation avec motif obligatoire, retour du statut à OUVERT, notification chef projet.	++
		US50	En tant que développeur, je souhaite démarrer un chronomètre sur un ticket afin de tracker mon temps.	Bouton Démarrer, création d'une session de travail, indicateur visuel de chrono actif.	+++
		US51	En tant que développeur, je souhaite mettre en pause mon chronomètre afin de gérer les interruptions.	Bouton Pause, enregistrement du temps écoulé, reprise possible.	++

		US52	En tant que développeur, je souhaite arrêter le chronomètre afin de finaliser ma session.	Arrêt avec commentaire optionnel, ajout de la session à l'historique, notification si dépassement.	+++
		US53	En tant que développeur, je souhaite ajouter manuellement une session de travail afin de corriger des oublis.	Formulaire avec date, heures, durée et commentaire obligatoire, validation sans chevauchement.	++
		US54	En tant que développeur, je souhaite changer le statut d'un ticket afin de refléter l'avancement.	Workflow OUVERT → EN_COURS → EN_REVUE → RESOLU → FERME, notification parties prenantes.	+++
		US55	En tant que client, je souhaite valider la résolution d'un ticket afin de confirmer que le problème est résolu.	Validation avec commentaire, fermeture du ticket, notification au développeur.	+++
		US56	En tant que chef de projet, je souhaite rouvrir un ticket fermé afin de traiter un problème récurrent.	Réouverture avec motif, réinitialisation du SLA, notification des assignés.	++
		US57	En tant que chef de projet, je souhaite consulter le temps passé par ticket afin d'analyser la productivité.	Temps total et détaillé par développeur avec graphiques.	++
		US58	En tant que développeur, je souhaite signaler un blocage afin d'alerter sur un obstacle.	Signalement avec type, description et impact, notification urgence au chef de projet, pause SLA.	+++
		US59	En tant que membre d'équipe, je souhaite résoudre un blocage afin de permettre au développeur de reprendre son travail.	Le blocage est résolu avec commentaire obligatoire, le chrono SLA reprend et l'action est tracée.	++
		US60	En tant que développeur, je souhaite demander une extension de délai SLA afin de disposer de plus de temps.	La demande d'extension est créée avec durée et justification pour validation par le chef de projet.	++

		US61	En tant que chef de projet, je souhaite approuver ou rejeter les demandes d'extension SLA.	La liste des demandes en attente est affichée, l'approbation ou rejet avec commentaire met à jour automatiquement le SLA.	++
		US62	En tant qu'utilisateur, je souhaite commenter un ticket avec mentions afin de communiquer efficacement.	Les commentaires sont ajoutés avec éditeur riche, autocomplétion @utilisateur et distinction internes/externes.	+++
		US63	En tant qu'utilisateur, je souhaite ajouter des pièces jointes aux tickets afin de fournir du contexte.	Les fichiers sont ajoutés par drag & drop (images/PDF/logs, max 10MB) avec prévisualisation.	++
		US64	En tant que développeur, je souhaite être informé lorsqu'un autre utilisateur modifie le même ticket.	Une notification temps réel via WebSocket affiche un toast et un indicateur "X est en train d'éditer".	++
		US65	En tant qu'utilisateur, je souhaite consulter l'historique complet d'un ticket afin de comprendre son évolution.	La timeline chronologique inversée affiche tous les événements avec filtres par type et export PDF.	++
10	Messagerie et collaboration	US66	En tant qu'utilisateur, je souhaite accéder à une messagerie intégrée afin de communiquer avec mon équipe.	La sidebar affiche les channels avec indicateurs de messages non lus et statut de présence des utilisateurs.	++
		US67	En tant qu'utilisateur, je souhaite envoyer des messages dans un channel afin de communiquer.	Les messages sont envoyés avec éditeur riche, mentions @, pièces jointes et livraison via WebSocket.	+++
		US68	En tant qu'utilisateur, je souhaite répondre à un message afin de créer un fil de discussion.	Un thread dédié est créé pour les réponses avec notifications uniquement aux participants.	++
		US69	En tant qu'utilisateur, je souhaite mentionner des collègues afin d'attirer leur attention.	L'autocomplétion @nom fonctionne, une notification push est envoyée et un badge mention non lue s'affiche.	+++

		US70	En tant qu'utilisateur, je souhaite éditer mes messages afin de corriger des erreurs.	L'édition est possible dans les 15min avec indicateur "(modifié)" et historique des modifications.	+
		US71	En tant qu'utilisateur, je souhaite supprimer mes messages afin de retirer du contenu inapproprié.	Le message est supprimé avec confirmation, remplacé par "[Message supprimé]" et conservé pour audit.	+
		US72	En tant qu'utilisateur, je souhaite épingler des messages importants afin de les retrouver facilement.	Les messages sont épinglés (permission requise) avec limite de 10 par channel et possibilité de désépinglage.	++
		US73	En tant que chef de projet, je souhaite voir qui est en ligne dans mon équipe afin de savoir qui je peux solliciter.	La liste des membres en ligne est affichée avec filtrage par projet/équipe et dernière activité.	++
		US74	En tant qu'utilisateur, je souhaite voir qui est en ligne afin de savoir qui est disponible.	L'indicateur de statut (EN_LIGNE, ABSENT, NE_PAS_DERANGER) est affiché et modifiable.	++
11	Gestion des documents	US75	En tant qu'utilisateur, je souhaite uploader des documents afin de centraliser les fichiers.	Les documents sont uploadés par drag & drop avec association à contrat/projet/ticket et métadonnées (titre, description, tags).	++
		US76	En tant qu'utilisateur, je souhaite télécharger des documents afin de les consulter hors ligne.	Le téléchargement individuel ou multiple (ZIP) est possible avec vérification des permissions et enregistrement de la consultation.	++
		US77	En tant qu'utilisateur, je souhaite gérer les versions des documents afin d'assurer la traçabilité.	Une nouvelle version est uploadée avec numérotation automatique (v1, v2) et commentaire obligatoire.	++
		US78	En tant qu'utilisateur, je souhaite consulter l'historique des consultations afin de voir qui a accédé au document.	L'historique des consultations est disponible pour traçabilité.	+

		US79	En tant qu'utilisateur, je souhaite archiver des documents afin de nettoyer la vue active.	Le document est archivé avec confirmation, les données sont conservées et la restauration est possible.	+
		US80	En tant qu'utilisateur, je souhaite être alerté si un fichier identique existe déjà afin d'éviter les doublons.	Le hash du fichier est calculé à l'upload, une alerte s'affiche si doublon avec options remplacer/créer version/annuler.	++
		US81	En tant que chef de projet, je souhaite générer automatiquement un PV à partir d'un ticket afin de documenter les interventions.	Un PDF professionnel est généré avec logo, en-tête, description, actions, temps passé et signature électronique.	+++
		US82	En tant que client, je souhaite télécharger les PV de mes tickets afin de conserver une trace des interventions.	Le PV est accessible depuis le ticket, l'historique est consultable et une notification email est envoyée à la génération.	++
12	Notifications et alertes	US83	En tant qu'utilisateur, je souhaite recevoir des notifications in-app afin d'être informé en temps réel.	L'icône avec compteur affiche un dropdown listant les notifications avec marquage lu/non lu et accès direct à l'élément.	+++
		US84	En tant qu'utilisateur, je souhaite recevoir des notifications par email afin d'être informé hors connexion.	Les préférences de fréquence (temps réel/digest) sont configurables avec templates professionnels.	++
		US85	En tant que membre technique, je souhaite recevoir des notifications Slack afin d'être alerté sur mon outil de travail.	La configuration Slack par contrat/projet est possible avec webhook URL et événements sélectionnables.	++
		US86	En tant qu'utilisateur, je souhaite recevoir des notifications push navigateur afin d'être alerté même hors onglet.	La permission est demandée, les notifications pour événements critiques sont envoyées et le clic ouvre l'application.	++
		US87	En tant qu'utilisateur, je souhaite consulter l'historique de mes notifications afin de retrouver une information.	La liste complète des notifications est affichée avec filtres par type/statut/période, recherche et pagination.	++

13	Dashboards et rapports	US88	En tant qu'administrateur, je souhaite consulter un dashboard global afin de piloter le système.	Les KPIs (utilisateurs actifs, tickets ouverts, taux SLA, CA) sont affichés avec graphiques, alertes et filtres par période.	+++
		US89	En tant que chef de projet, je souhaite consulter un dashboard d'équipe afin de piloter l'activité.	La vue d'équipe affiche la charge de travail par développeur, performance SLA et temps passé vs estimé.	+++
		US90	En tant que développeur, je souhaite consulter mon dashboard personnel afin d'organiser ma journée.	Les tickets assignés sont affichés avec priorités, chronomètre actif, temps passé et échéances SLA.	++
		US91	En tant que client, je souhaite consulter mon dashboard afin de suivre mes projets.	Les contrats actifs, demandes et tickets en cours, statistiques, documents récents et contacts équipe sont affichés.	++
		US92	En tant qu'administrateur, je souhaite générer un rapport d'activité mensuel afin de présenter les statistiques.	Le rapport contient tickets traités, temps passé et respect SLA avec export PDF/Excel et graphiques.	+++
		US93	En tant qu'administrateur, je souhaite générer des rapports de conformité SLA afin de justifier les performances.	Le rapport affiche le taux de respect par contrat/période avec détail par ticket, calcul pénalités et PDF professionnel.	+++
		US94	En tant que chef de projet, je souhaite générer des rapports d'activité afin de présenter l'avancement.	Le rapport par projet/équipe/période affiche tickets traités, temps passé et graphiques d'évolution.	++
14	Historique et traçabilité	US95	En tant qu'administrateur, je souhaite consulter l'historique complet de toutes les actions afin d'assurer la traçabilité.	Toutes les actions CRUD sont enregistrées avec utilisateur, date, IP, action, entité et valeurs modifiées.	+++

		US96	En tant qu'administrateur, je souhaite exporter les logs d'audit afin de répondre aux exigences de conformité.	L'export CSV/JSON est possible avec filtrage par période/utilisateur/action, signature numérique et chiffrement.	++
15	Versioning avancé	US97	En tant qu'utilisateur, je souhaite créer une nouvelle version d'un document afin de maintenir l'historique.	Une nouvelle version est uploadée avec commentaire obligatoire, numérotation auto (v1.0, v1.1, v2.0) et conservation des versions précédentes.	++
		US98	En tant qu'utilisateur, je souhaite comparer deux versions d'un document afin de voir les différences.	Deux versions sont sélectionnées et affichées côte à côte avec mise en évidence des modifications.	+
		US99	En tant qu'administrateur, je souhaite voir qui a consulté un document afin de suivre sa diffusion.	La liste des consultations affiche date/heure/utilisateur et durée de consultation avec statistiques.	++
16	Assistance intelligente	US100	En tant que client, je souhaite que le système suggère automatiquement une catégorie pour ma demande.	Le titre et la description sont analysés, une catégorie est suggérée et le client peut l'accepter ou la modifier.	+++
		US101	En tant qu'administrateur, je souhaite que le modèle de suggestion s'améliore afin d'offrir des recommandations précises.	Le modèle apprend des validations/rejets, se réentraîne périodiquement et les métriques de précision sont suivies.	++
		US102	En tant que chef de projet, je souhaite que le système suggère une priorité afin de traiter les demandes efficacement.	Le contenu et le contexte client sont analysés, une priorité est suggérée et ajustable par le chef de projet.	++
		US103	En tant que développeur, je souhaite des suggestions de solutions basées sur le passé afin de résoudre plus rapidement.	Les tickets similaires sont identifiés, les solutions précédentes sont suggérées et peuvent être acceptées ou ignorées.	++

		US104	En tant que développeur, je souhaite évaluer la pertinence des suggestions afin d'améliorer la qualité.	Un système de notation (utile/pas utile) collecte les retours et ajuste le modèle.	+
		US105	En tant qu'administrateur, je souhaite que le système détecte automatiquement les doublons afin d'éviter les demandes redondantes.	La similarité est analysée à la création, les doublons sont signalés et les demandes similaires sont liées automatiquement.	++
17	Gestion des sous-tâches	US106	En tant qu'utilisateur, je souhaite créer des sous-tâches pour décomposer un ticket afin de mieux organiser le travail.	Les sous-tâches sont créées avec titre/description/estimation et la progression globale est calculée automatiquement.	++
		US107	En tant que chef de projet, je souhaite assigner des sous-tâches à différents développeurs afin de paralléliser le travail.	Chaque sous-tâche est assignée indépendamment, les développeurs sont notifiés et le temps individuel est suivi.	++
		US108	En tant qu'utilisateur, je souhaite suivre la progression via les sous-tâches afin de visualiser l'avancement détaillé.	La barre de progression affiche le statut de chaque sous-tâche et identifie les sous-tâches bloquantes.	++
		US109	En tant que système, je souhaite fermer automatiquement le ticket parent lorsque toutes les sous-tâches sont terminées.	La vérification est automatique à chaque changement de statut, le ticket parent passe à RESOLU et le client est notifié.	++

3.2 Planification des sprints

Un sprint représente une période de développement durant laquelle l'équipe s'engage à livrer un ensemble cohérent de fonctionnalités opérationnelles. Cette approche itérative permet d'obtenir des retours réguliers et d'ajuster les priorités au fil de l'avancement.

Notre projet est organisé en 4 sprints de 3 à 4 semaines chacun, couvrant une période totale de développement de 3,5 mois. Chaque sprint commence par une réunion de planification (Sprint Planning) où l'équipe sélectionne les user stories du Product Backlog à réaliser, et se termine par une revue de sprint (Sprint Review) et une rétrospective (Sprint Retrospective).

La figure 2.4 illustre le déroulement chronologique de nos sprints.

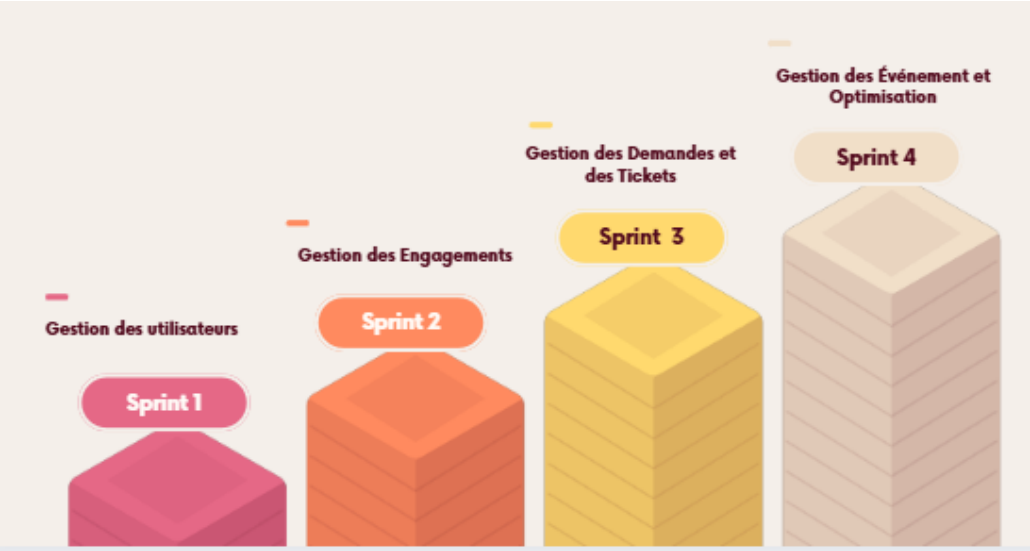

FIGURE 2.4 – Planification chronologique des sprints

3.2.1 Sprint 1 - Gestion des Utilisateurs et Authentification

Période : du 17 février au 11 mars (3 semaines)

Objectif du sprint : Mettre en place le système d'authentification sécurisé et la gestion complète des utilisateurs avec contrôle d'accès basé sur les rôles (RBAC).

User Stories sélectionnées :

- **Epic 1 - Authentification et Sécurité** : US01 à US05 (5 user stories)
- **Epic 2 - Gestion des utilisateurs** : US06 à US11 (6 user stories)
- **Epic 3 - Gestion des rôles et permissions** : US12 à US15 (4 user stories)
- **Epic 4 - Demandes d'inscription OAuth** : US16 à US17 (2 user stories)

Livrables attendus :

- Système d'authentification fonctionnel avec support OAuth
- Module de gestion des utilisateurs opérationnel
- Système RBAC complet avec gestion des rôles et permissions

3.2.2 Sprint 2 - Gestion des Clients et Contrats

Période : du 12 mars au 2 avril (3 semaines)

Objectif du sprint : Développer la gestion complète des clients, projets et contrats de maintenance avec système d'alertes.

User Stories sélectionnées :

- **Epic 5 - Gestion des clients** : US18 à US24 (7 user stories)
- **Epic 6 - Gestion des projets** : US25 à US28 (4 user stories)
- **Epic 7 - Gestion des contrats** : US29 à US38 (10 user stories)
- **Epic 14 - Historique et traçabilité** : US95 à US96 (2 user stories)

Livrables attendus :

- Module de gestion des clients et projets

- Système de gestion des contrats avec SLA et alertes
- Système d'audit et traçabilité complet

3.2.3 Sprint 3 - Gestion des Demandes et Tickets

Période : du 3 avril au 1 mai (4 semaines)

Objectif du sprint : Implémenter le cœur métier avec la gestion des demandes et tickets incluant suivi du temps et SLA.

User Stories sélectionnées :

- **Epic 8 - Gestion des demandes** : US39 à US46 (8 user stories)
- **Epic 9 - Gestion des tickets** : US47 à US66 (20 user stories)
- **Epic 17 - Gestion des sous-tâches** : US106 à US110 (4 user stories)
- **Epic 13 - Dashboards** : US88 à US91 (4 user stories)

Livrables attendus :

- Module de gestion des demandes de maintenance
- Système complet de gestion des tickets avec Kanban
- Suivi du temps et respect des SLA
- Dashboards pour pilotage

3.2.4 Sprint 4 - Collaboration, Documents et IA

Période : du 2 mai au 31 mai (4 semaines)

Objectif du sprint : Finaliser avec les fonctionnalités avancées de collaboration, gestion documentaire et IA.

User Stories sélectionnées :

- **Epic 10 - Messagerie** : US67 à US75 (9 user stories)
- **Epic 11 - Gestion des documents** : US76 à US83 (8 user stories)
- **Epic 12 - Notifications** : US84 à US88 (5 user stories)
- **Epic 13 - Rapports** : US92 à US94 (3 user stories)
- **Epic 15 - Versioning avancé** : US97 à US99 (3 user stories)
- **Epic 16 - Intelligence Artificielle** : US100 à US105 (6 user stories)

Livrables attendus :

- Système de messagerie et collaboration temps réel
- Module de gestion documentaire avec versioning
- Système de notifications multicanaux
- Module de génération de rapports
- Moteur d'IA pour assistance intelligente

4 Environnement de Développement

On va commencer par la présentation de l'environnement matériel et logiciel que nous utilisons.

4.1 Environnement matériel

Ordinateur	1	2
propriétés	Rihab Chourou	Eya Gharbi
Processeur	Core i3	Core i3
Ram	20 Go	16 Go
Disque Dure	1 To	256 Go
Système d'exploitation	Windows 10 Professionnel N 64-bit	Windows 10 Professionnel 64-bit

TABLE 2.2 – Environnement de développement

4.2 Environnement logiciel

4.2.1 Langage de programmation et framework

Next V16.2.2 : « C'est un framework React permettant de créer des applications web complètes. Il configure également automatiquement les outils de bas niveau tels que les modules de regroupement et les compilateurs. » |6|

NestJs V11.0.16 : « Cest un framework permettant de créer des applications serveur Node.js performantes et évolutives. Il utilise JavaScript progressif, est conçu avec TypeScript et le prend entièrement en charge (tout en permettant aux développeurs de coder en JavaScript pur). » |7|

Tailwind CSS V4.2.1 : « Un framework CSS utilitaire, doté de classes telles que flex , pt-4 , text-center et rotate-90, qui peuvent être combinées pour créer n'importe quel design, directement dans votre balisage. » |8|

shadcn/ui V3.8.5 : « propose une vaste collection de composants aux styles par défaut soigneusement sélectionnés. Ils sont conçus pour être esthétiques individuellement et pour fonctionner harmonieusement ensemble au sein d'un système cohérent. » |9|

4.2.2 Outils de développement et modélisation

Visual Studio Code : « Visual Studio est un puissant environnement de développement intégré (IDE) pour Windows où vous pouvez développer, générer, déboguer, tester et déployer vos applications, dans un seul endroit. Il inclut des compilateurs, des outils d'achèvement du code, un contrôle de code source. » |10|

GitLab : « une plateforme qui personnalise le flux de travail pour le développement, les tests, la sécurité et le déploiement. Les équipes orchestrent le cycle de vie du logiciel, GitLab s'adapte à l'environnement réglementaire tout en préservant la réactivité dont les équipes ont besoin. » |11|

UML : « est un langage graphique permettant de visualiser, de spécifier et de construire les artefacts d'un système logiciel complexe. UML offre une méthode standard pour concevoir les plans d'un système. » |12|

draw.io : « est une pile technologique permettant de créer des applications de diagrammes, et le logiciel de diagrammes utilisateur final basé sur navigateur le plus utilisé au monde. » |13|

Swagger : « est un ensemble d'outils open source basés sur la spécification OpenAPI qui permettent de concevoir, de créer, de documenter et d'utiliser des API REST. » |14|

4.2.3 Systèmes de Gestion de Bases de données

Prisma ORM V7.4.2 : « est un ORM open source qui offre un accès rapide et sécurisé aux bases de données Postgres, MySQL et autres, et fonctionne parfaitement sur Node.js. » |15|

PostgreSQL : « est un système de gestion de bases de données objet-relationnelles (SGBD-OR). Il a été pionnier dans de nombreux concepts qui ne sont devenus disponibles que bien plus tard dans certains systèmes de bases de données commerciaux. » |16|

Neon : « est la couche Postgres pour Internet : une version entièrement gérée, sans serveur et open source de Postgres, conçue pour aider les développeurs à créer des applications évolutives et fiables plus rapidement que jamais. Neon repose sur une architecture distribuée native du cloud qui sépare le stockage et le calcul, offrant ainsi à Postgres l'évolutivité, la fiabilité et l'efficacité requises par les applications modernes. » |17|

4.2.4 Outil de Gestion de Projet Scrum

Notion : « est un puissant outil no-code de productivité. Il aide à gérer une base de connaissances, pour rendre plus facile et convivial le travail au sein des collaborateurs. Cette application se révèle indispensable autant pour les travailleurs autonomes que pour les professionnels en entreprise. C'est un également un outil parfait pour gérer son side project. » |18|

4.2.5 Environnement de composition de documents

LaTeX : « est un système de composition de haute qualité ; il comprend des fonctionnalités conçues pour la production de documents techniques et scientifiques. LaTeX est la norme de facto pour la communication et la publication de documents scientifiques. » |19|

4.2.6 Outil de design graphique

Canva : « est un outil de conception et de publication en ligne dont la mission est de permettre à chacun dans le monde de concevoir n'importe quel contenu et de le publier n'importe où. » |20|

5 Architecture de l'application

5.1 Architecture physique

L'architecture système est un ensemble de décisions de conception qui définit l'interaction des composants entre eux. Le choix de l'architecture est très sensible car elle permet d'optimiser les performances du système et facilite l'évolution du système en fonction des volumes de données. La figure ci-dessous montre l'architecture de notre application :

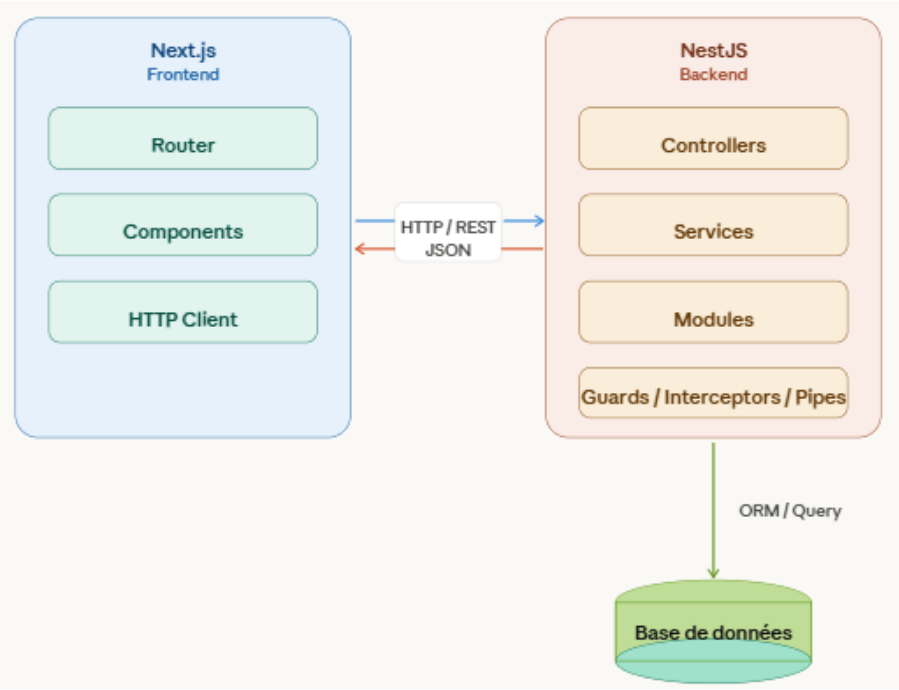

FIGURE 2.5 – architecture physique

5.2 Architecture logique

Nous avons choisi comme architecture logicielle l'architecture Modulaire qui consiste à diviser un système en plusieurs modules. Elle influence donc l'ensemble architectural de notre application.
Nous avons opté pour cette architecture afin de faciliter l'intégration de nouvelles fonctionnalités au système en ajoutant simplement des modules supplémentaires. On peut tester les modules individuellement, ce qui permet d'identifier rapidement les erreurs.

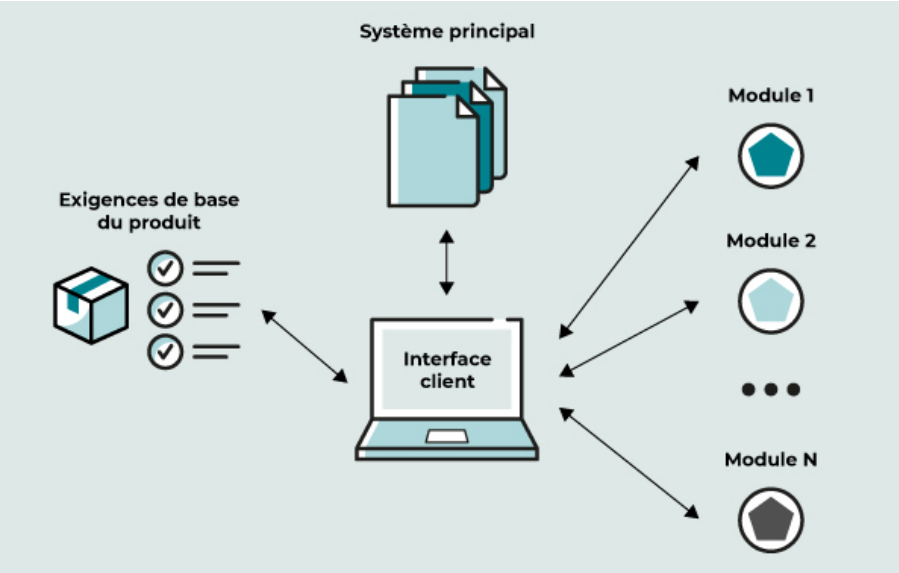

FIGURE 2.6 – Architecture Modulaire

Chapitre 3. Sprint 1 : Gestion des Utilisateurs et Authentification

Plan

1 Backlog du Sprint 1 44

2 Analyse 50

3 Conception 56

4 Réalisation 57

5 Tests 58

Introduction

Dans ce chapitre, nous abordons le premier sprint du projet SolidMaint, couvrant la période du 17 février au 11 mars 2026, soit trois semaines de travail consacrées à la mise en place des fondations de l'application. Nous commencerons par la présentation du backlog du sprint regroupant dix-sept user stories réparties en quatre epics, suivie de la phase d'analyse et de conception portant sur le système d'authentification et la gestion des utilisateurs par rôles (RBAC), puis de la phase de réalisation des interfaces et des choix d'implémentation, et enfin des tests conduits pour valider les fonctionnalités livrées.

Sprint Backlog

Le tableau ci-dessous présente le Sprint Backlog détaillé avec les descriptions techniques et l'effort estimé pour chaque user story.

ID US	User Story	Description technique	Effort (j)
Epic 1 - Authentification et Sécurité			
US01	En tant qu'utilisateur, je souhaite m'authentifier avec email et mot de passe afin d'accéder à mes informations de manière sécurisée.	• Développer l'API du Login • Mettre en place la gestion de session (JWT) • Réaliser l'interface connexion login • Consommer l'API d'authentification • Gérer les tentatives échouées (3 essais) avec verrouillage temporaire	1
US02	En tant qu'utilisateur, je souhaite réinitialiser mon mot de passe afin de retrouver l'accès à mon compte.	• Créer les formulaires "mot de passe oublié" et de réinitialisation • Développer les API associées • Générer un token sécurisé avec expiration • Configurer l'envoi d'email de confirmation	2

US03	En tant qu'utilisateur, je souhaite m'authentifier via Google ou GitHub afin de me connecter rapidement sans créer de compte.	• Intégrer les stratégies OAuth2 via Google et GitHub dans NestJS en utilisant Passport (@nestjs/passport) • Configurer les stratégies avec récupération et validation des données du profil utilisateur • Vérifier l'existence de l'utilisateur en base de données et une demande d'inscription OAuth est envoyée à l'admin • Implémenter les routes /auth/google, /auth/github et leurs callbacks, avec génération d'un JWT • Implémenter les boutons OAuth sur la page de login	2
US04	En tant qu'utilisateur, je souhaite me déconnecter de la plateforme afin de sécuriser ma session.	• Développer l'endpoint POST /auth/logout avec révocation du refresh token en base • Supprimer les tokens et données utilisateur du localStorage • Réaliser le bouton de déconnexion	2
US05	En tant qu'utilisateur, je souhaite que mon compte soit protégé contre les tentatives de connexion malveillantes afin de garantir la sécurité de mes données.	• Implémenter un système de rate limiting avec compteur de tentatives • Développer la logique de verrouillage temporaire après 3 tentatives échouées • Configurer l'envoi automatique d'email de notification via Nodemailer • Stocker les tentatives en base avec timestamp et IP	2
Epic 2 - Gestion des utilisateurs			
US06	En tant qu'utilisateur, je souhaite gérer mon profil afin que mes informations soient toujours à jour.	• Développer les endpoints GET /profil et PUT /profil pour consulter et modifier les informations • Implémenter l'upload de la photo de profil POST /profil/upload-photo • Développer le changement de mot de passe avec vérification de l'ancien • Réaliser l'interface profil côté Next.js avec formulaire prérempli	2

US07	En tant qu'administrateur, je souhaite créer des utilisateurs afin de gérer les accès au système.	• Développer l'endpoint POST /users pour créer un compte avec la génération automatique d'un mot de passe temporaire hashé via bcrypt • Valider les données d'entrée via class-validator • Réaliser le formulaire d'ajout côté Next.js • Envoyer un e-mail de bienvenue via Nodemailer avec les identifiants générés	2
US08	En tant qu'administrateur, je souhaite modifier les informations d'un utilisateur afin de maintenir des données à jour.	• Développer l'endpoint PUT /users/ :id • Sécuriser l'accès à l'endpoint pour les administrateurs uniquement • Valider les données via class-validator • Réaliser le formulaire de modification	2
US09	En tant qu'administrateur, je souhaite activer et désactiver un compte utilisateur afin de contrôler l'accès au système.	• Développer les endpoints PATCH /users/ :id/desactiver et PATCH /users/ :id/reactiver • Implémenter les vérifications de réassignation des tickets et des demandes en cas de désactivation • Envoyer des notifications pour informer l'utilisateur de l'activation ou désactivation de son compte • Réaliser les boutons d'action avec appels des API	2
US10	En tant qu'administrateur, je souhaite consulter la liste des utilisateurs afin de visualiser et gérer efficacement les comptes.	• Développer l'endpoint GET /users retournant la liste des utilisateurs • Réaliser l'interface d'administration avec gestion	2
US11	En tant qu'utilisateur, je souhaite consulter l'historique de mes activités afin de suivre mes actions dans le système.	• Développer l'endpoint GET /users/ :id/activities pour récupérer l'historique • Implémenter la logique de filtrage dans le backend en utilisant l'opérateur contains • Implémenter une barre de recherche dynamique • Réaliser l'interface timeline avec filtres par date et type d'action	2
Epic 3 - Gestion des rôles et permissions			

US12	En tant qu'administrateur, je souhaite gérer des rôles afin de structurer les accès au système.	• Développer les endpoints CRUD pour la gestion complète des rôles • Implémenter le PermissionsGuard et le décorateur @Permissions pour sécuriser les routes du backend • Réaliser l'interface de gestion sous forme de matrice	2
US13	En tant qu'administrateur, je souhaite attribuer des permissions granulaires aux rôles afin de contrôler précisément les accès.	• Développer l'endpoint PATCH /roles-permissions/roles/ :id/sync pour synchroniser dynamiquement les associations entre un rôle et ses permissions. • Implémenter la logique de mise à jour atomique • Développer l'interface interactive de type "Matrice de contrôle d'accès"	2
US14	En tant qu'administrateur, je souhaite consulter la matrice rôles-permissions afin de visualiser rapidement les droits de chaque rôle.	• Développer l'endpoint GET /roles-permissions/matrix) pour récupérer les données • Implémenter la logique de structuration des données au format JSON • Réaliser l'interface de visualisation de type "Heatmap" ou "Tableau de bord"	2
Epic 4 - Demandes d'inscription OAuth			
US15	En tant qu'utilisateur externe, je souhaite demander l'accès via OAuth afin de m'inscrire facilement.	• Développer l'endpoint POST /auth/oauth/request pour créer une demande • Enregistrer la demande en base avec statut EN_ATTENTE • Implémenter l'envoi de notification à l'administrateur via email et in-app • Réaliser l'interface de demande d'accès	1
US16	En tant qu'administrateur, je souhaite valider ou rejeter les demandes d'inscription OAuth afin de contrôler les accès.	• Développer les endpoints PATCH /auth/oauth/requests/ :id/approve et /reject • Implémenter la création automatique du compte utilisateur lors de l'approbation • Configurer l'envoi d'emails de confirmation ou rejet via Nodemailer • Réaliser l'interface de gestion des demandes avec actions d'approbation/rejet	2

Epic 5 - Gestion Clients			
US17	En tant qu'administrateur, je souhaite créer un profil client afin de centraliser ses informations.	• Développer l'endpoint POST /clients pour orchestrer la création simultanée du compte CLIENT et du profil entreprise associé. • Implémenter un service de récupération automatique de logo (LogoService) établi sur le nom de la société • Intégrer une logique de gestion des projets historiques pour associer des projets passés, des chefs de projets et des membres d'équipe au nouveau client • Réaliser l'interface de formulaire	2
US18	En tant qu'administrateur, je souhaite consulter la liste des clients afin de visualiser les clients existants.	• Développer l'endpoint GET /clients intégrant une logique de filtrage RBAC. • Réaliser une interface de visualisation	2
US19	En tant qu'administrateur, je souhaite consulter le profil détaillé d'un client afin de visualiser toutes ses informations.	• Développer l'endpoint GET /clients/ :id pour récupérer les informations du client. • Réaliser une interface de profil structurée. • Développer des composants spécifiques pour l'affichage des informations secondaires	2
US20	En tant qu'administrateur, je souhaite modifier les informations d'un client afin de maintenir des données à jour.	• Développer l'endpoint PATCH /clients/ :id permettant la mise à jour des informations des clients. • Développer des contrôles d'intégrité côté serveur pour valider les formats de données. • Réaliser une interface d'édition fluide.	2
US21	En tant qu'administrateur, je souhaite activer et désactiver un compte client afin de contrôler l'accès.	• Développer les endpoints de changement de statut (POST /desactiver, POST /reactiver) • Implémenter la vérification pour bloquer l'opération si le client possède des contrats actifs, des demandes en attente ou des tickets non résolus. • Intégrer le EmailService pour l'envoi automatique de notifications transactionnelles de changement de statut.	2

US22	En tant qu'administrateur, je souhaite ajouter des notes privées sur un client afin de conserver des informations internes.	• Développer un endpoint POST /clients/ :id/notes permettant d'ajouter es chaînes de caractères dans le tableau notesPrivees. • Réaliser une interface de type "Bloc-notes" ou "Journal interne" dans le profil client. • Sécuriser l'accès à ces informations au niveau du backend pour garantir que ces notes ne soient jamais exposées au client lui-même	2
US23	En tant qu'administrateur, je souhaite imprimer l'annuaire d'un client afin de disposer d'une version papier.	• Développer l'endpoint POST /clients pour créer un client avec validation des données	2
Epic 5 - Gestion des projets et équipes			
US24	En tant qu'administrateur, je souhaite créer un projet rattaché à un client afin d'organiser les interventions.	• Développer l'endpoint POST /clients pour créer un client avec validation des données	2
US25	En tant qu'administrateur, je souhaite consulter la liste des projets afin de piloter l'activité.	• Développer l'endpoint POST /clients pour créer un client avec validation des données	2
US26	En tant qu'administrateur, je souhaite affecter des développeurs à un projet afin de constituer l'équipe.	• Développer l'endpoint POST /clients pour créer un client avec validation des données	2
US27	En tant que membre d'équipe, je souhaite consulter les informations du projet afin de comprendre le contexte.	• Développer l'endpoint POST /clients pour créer un client avec validation des données	2
		Total Sprint 1 :	**34 j**

1 Analyse

1.1 Diagramme des cas d'utilisation du premier sprint

La figure ci-dessous montre le diagramme de cas d'utilisation du premier sprint :

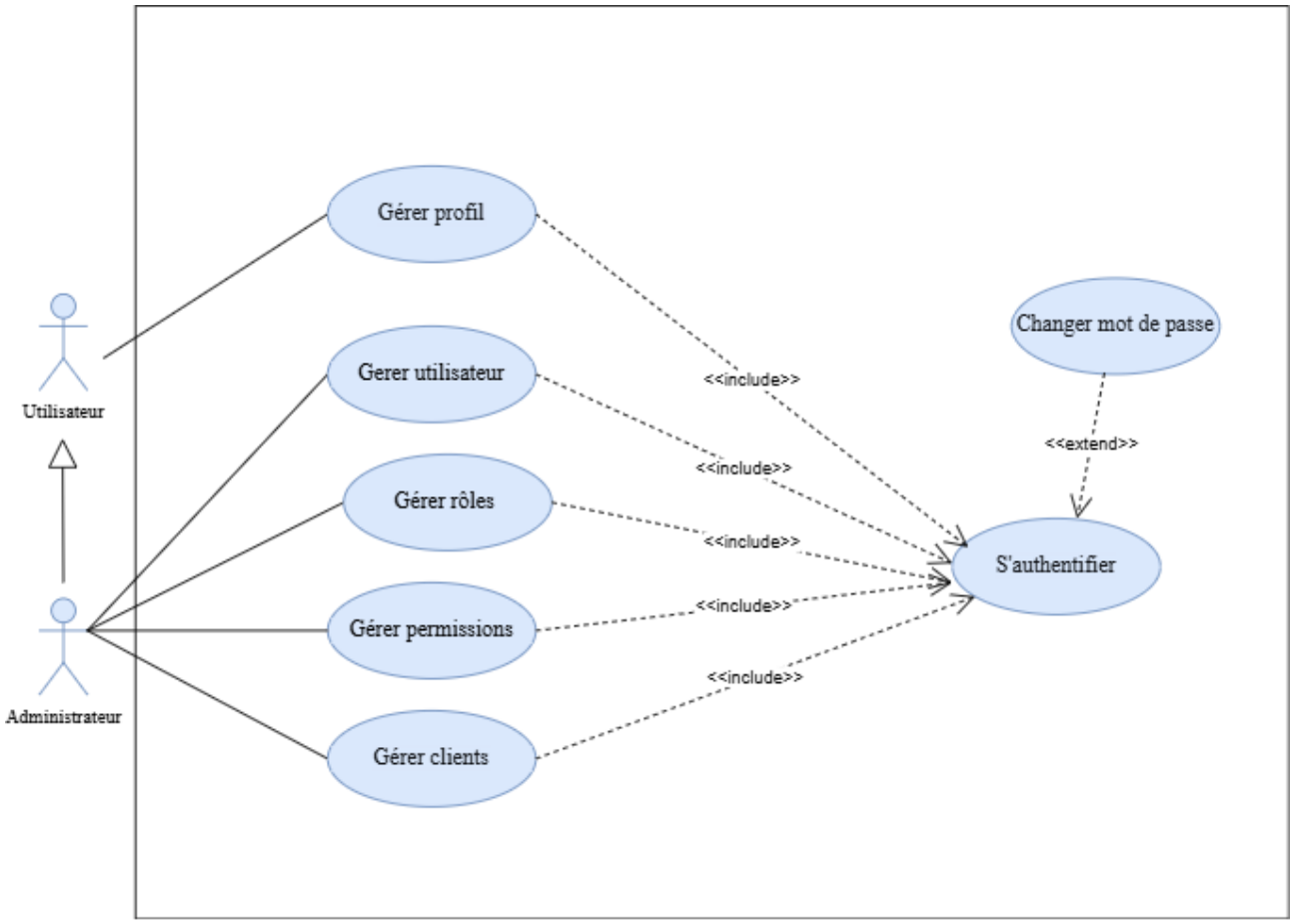

FIGURE 3.1 – Diagramme des cas d'utilisation du premier sprint

Nous allons accompagner les cas d'utilisation d'un texte descriptif pour expliquer le scénario de chaque cas ainsi que ses exceptions.

1.2 Description textuelle de cas d'utilisation « Changer mot de passe »

TABLE 3.2 – Description textuelle de cas d'utilisation « Changer mot de passe »

Nom du cas d'utilisation	Changer mot de passe
Acteurs	Utilisateurs
Préconditions	Utilisateur possède déjà un compte
Postconditions	Le mot de passe est réinitialisé
Scénario principal	1. Le système affiche un formulaire de changement de mot de passe. 2. L'utilisateur saisit son e-mail. 3. L'utilisateur clique sur le bouton « Réinitialiser le mot de passe ». 4. Le système vérifie l'existence de l'email. 5. Le système envoie un e-mail à l'utilisateur. 6. L'utilisateur clique sur le lien reçu par e-mail. 7. Le système affiche le formulaire de réinitialisation de mot de passe. 8. L'utilisateur saisit son nouveau mot de passe deux fois pour confirmation. 9. Le système valide et enregistre le nouveau mot de passe. 10. Le système envoie un e-mail de confirmation à l'utilisateur.
Scénarios d'exception	• L'utilisateur saisit un e-mail qui ne correspond à aucun compte existant. • L'utilisateur a rempli un champ invalide, ou a laissé un champ vide. • Le lien de réinitialisation est expiré. • L'utilisateur ne confirme pas correctement le mot de passe.

1.3 Description des cas d'utilisation « Gérer utilisateurs »

1.3.1 Raffinement de cas d'utilisation « Gérer utilisateurs »

Au cours de cette partie, nous allons analyser chaque cas d'utilisation raffiné pour le cas de gestion des utilisateurs :

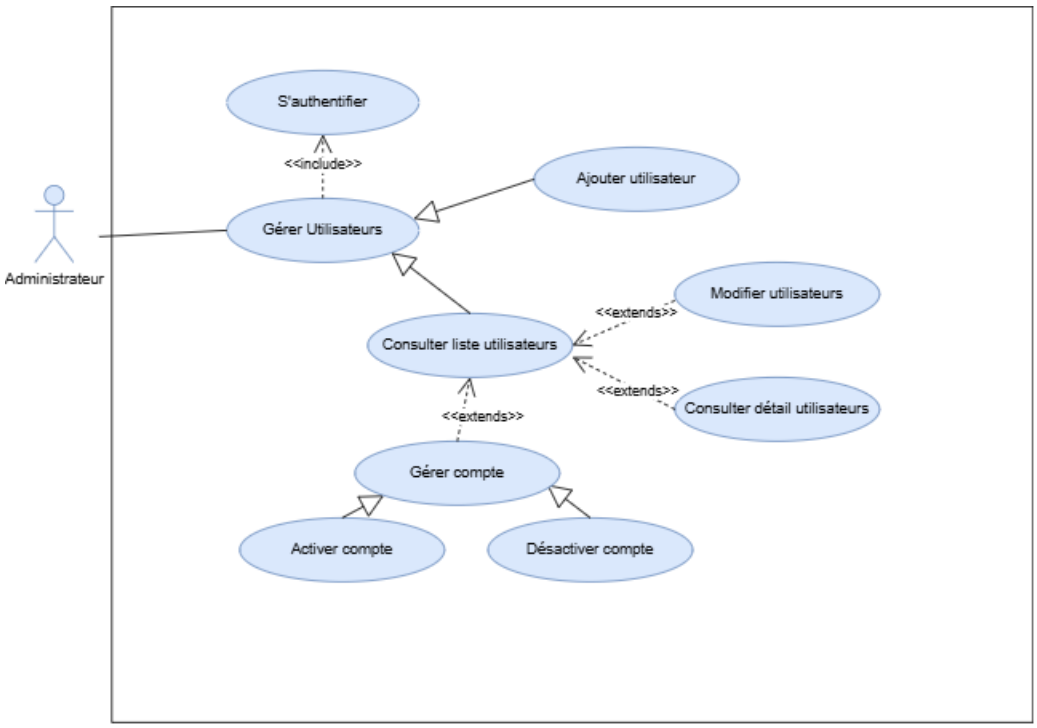

FIGURE 3.2 – Diagramme des cas d'utilisation « Gérer utilisateurs »

1.3.2 Description textuelle de cas d'utilisation « Consulter liste utilisateurs »

TABLE 3.3 – Description textuelle « Consulter liste utilisateurs »

Nom du cas d'utilisation	Consulter liste utilisateurs
Acteurs	Administrateur
Préconditions	Utilisateur authentifié
Postconditions	Liste des utilisateurs affichée
Scénario principal	1. L'administrateur accède à la section "Utilisateurs". 2. Le système affiche les utilisateurs sous forme de grille (par défaut) ou de liste selon le choix de l'utilisateur. 3. L'administrateur recherche un utilisateur en saisissant des informations clés dans la barre de recherche ou par des boutons de filtre (par statut, par rôle). 4. Le système filtre la liste.
Scénarios alternatifs	L'administrateur clique sur le bouton « Exporter ».

1.3.3 Description textuelle de cas d'utilisation « Ajouter utilisateur »

TABLE 3.4 – Description textuelle « Ajouter utilisateur »

Nom du cas d'utilisation	Ajouter utilisateur
Acteurs	Administrateur
Préconditions	Utilisateur authentifié Liste des utilisateurs affichée
Postconditions	Utilisateur est ajouté
Scénario principal	1. L'administrateur clique sur le bouton "Nouvel Utilisateur". 2. Le système affiche le formulaire d'ajout. 3. L'administrateur remplit le formulaire. 4. L'administrateur valide le formulaire en cliquant sur (Créer le compte). 5. Le système affiche le message de succès.
Scénarios d'exception	• L'administrateur a rempli un champ invalide, ou a laissé un champ vide. • L'administrateur saisit un e-mail déjà utilisé dans un autre compte. • L'administrateur tente de fermer la fenêtre alors qu'il a commencé à saisir des données.
Scénarios alternatifs	Le système génère un mot de passe temporaire.

1.3.4 Description textuelle de cas d'utilisation « Modifier utilisateur »

TABLE 3.5 – Description textuelle « Modifier utilisateur »

Nom du cas d'utilisation	Modifier utilisateur
Acteurs	Administrateur
Préconditions	Utilisateur authentifié Liste des utilisateurs affichée
Postconditions	L'utilisateur est modifié
Scénario principal	1. L'administrateur consulte le profil de l'utilisateur choisi. 2. L'administrateur clique sur le bouton (Modifier). 3. Le système affiche le formulaire de modification. 4. L'administrateur modifie les informations souhaitées. 5. L'administrateur clique sur le bouton (Enregistrer). 6. Le système fait la mise à jour.
Scénarios alternatifs	L'administrateur supprime leur dernier rôle. L'utilisateur tente d'ajouter un rôle qui ne correspond pas au parcours métier.

1.3.5 Description textuelle de cas d'utilisation « Désactiver un utilisateur »

TABLE 3.6 – Description textuelle « Désactiver un utilisateur »

Nom du cas d'utilisation	Désactiver un utilisateur
Acteurs	Administrateur
Préconditions	Utilisateur authentifié Liste des utilisateurs affichée L'utilisateur à désactiver doit être actuellement Actif
Postconditions	Le compte est désactivé
Scénario principal	1. L'administrateur choisit un compte dans la liste. 2. L'administrateur clique sur l'action (Désactiver). 3. Le système vérifie si l'utilisateur possède des tickets actifs ou des demandes en attente. 4. Le système affiche une boîte de dialogue de confirmation. 5. L'administrateur confirme l'action. 6. Le système met à jour le statut de l'utilisateur. 7. Le système envoie un e-mail automatique à l'utilisateur pour l'informer de la suspension de son compte.
Scénarios alternatifs	L'administrateur réassigne les tickets ou les demandes.
Scénarios d'exception	• L'utilisateur tente de désactiver un utilisateur qui possède des tickets actifs ou des demandes en attente.

1.4 Description des cas d'utilisation « Gérer profil »

1.4.1 Raffinement de cas d'utilisation « Gérer Profil »

Au cours de cette partie, nous allons analyser chaque cas d'utilisation raffiné pour le cas de gestion de profil :

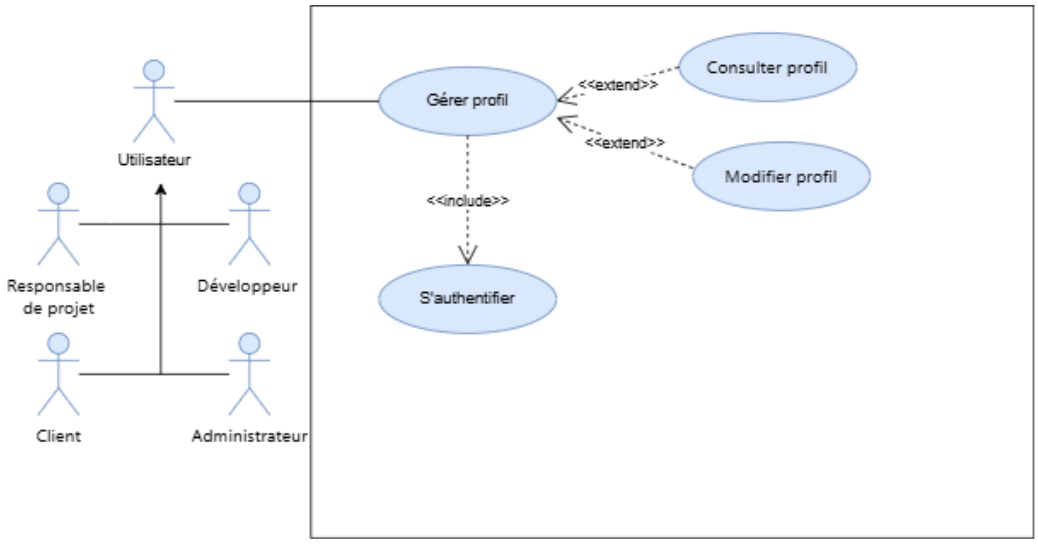

FIGURE 3.3 – Raffinement de cas d'utilisation « Gérer Profil »

1.4.2 Description textuelle de cas d'utilisation « Consulter Profil »

TABLE 3.7 – Description textuelle « Consulter profil »

Nom du cas d'utilisation	Consulter profil
Acteurs	Utilisateur
Préconditions	Utilisateur authentifié
Postconditions	Le profil choisi est consulté
Scénario principal pour consulter son propre profil	1. L'utilisateur clique sur (Voir le profil). 2. Le système affiche la page personnelle. 3. L'utilisateur consulte ses données et son historique de modification.
Consulter le profil d'un utilisateur	1. L'administrateur choisit un utilisateur dans la liste puis clique sur (Voir profil). 2. Le système affiche les informations de l'employé.

1.4.3 Description textuelle de cas d'utilisation « Modifier Profil »

TABLE 3.8 – Description textuelle « Modifier le Profil »

Nom du cas d'utilisation	Modifier le Profil
Acteurs	Administrateur, Utilisateur
Préconditions	Utilisateur authentifié
Postconditions	Le Profil est modifié
Scénario principal pour Modifier le profil d'un utilisateur	1. L'administrateur accède au profil de l'utilisateur. 2. L'administrateur modifie les accès en ajoutant et retirant des rôles système. 3. L'administrateur change le statut du compte (désactiver ou activer). 4. Le système valide chaque action.
Scénario principal pour Modifier son propre profil	1. L'utilisateur accède à sa page "Mon profil". 2. L'utilisateur met à jour sa photo de profil ou ses coordonnées. 3. L'utilisateur change son mot de passe en saisissant son ancien mot de passe pour validation. 4. L'utilisateur ajoute ou définit un nouvel e-mail et choisit lequel serait le principal. 5. Le système enregistre les changements.
Scénarios d'exception	• L'administrateur veut retirer le dernier rôle. • L'utilisateur veut modifier ses propres rôles.

2 Conception

2.1 Diagrammes de séquence

Cette section présentera les diagrammes de séquence détaillant les interactions entre les différents composants du système pour les cas d'utilisation principaux du Sprint 1.

[À compléter avec les diagrammes de séquence]

2.2 Modèle de données

Cette section présentera le modèle de données relatif à la gestion des utilisateurs, rôles et permissions.

[À compléter avec le schéma de base de données]

3 Réalisation

3.1 Architecture technique

Cette section présentera l'architecture technique mise en place pour le Sprint 1, incluant la structure des modules frontend et backend.

[À compléter avec les détails d'implémentation]

3.2 Interfaces utilisateur

Cette section présentera les principales interfaces développées durant le Sprint 1.

[À compléter avec les captures d'écran]

4 Tests

4.1 Tests unitaires

Cette section présentera les tests unitaires réalisés pour valider les fonctionnalités du Sprint 1.

[À compléter avec les résultats des tests]

4.2 Tests d'intégration

Cette section présentera les tests d'intégration effectués pour s'assurer du bon fonctionnement global du système.

[À compléter avec les résultats des tests]

Chapitre 4. Sprint 2 : Gestion des Clients et Contrats

Plan

1 Backlog du Sprint 2 60

2 Analyse 63

3 Conception 64

4 Réalisation 65

5 Tests 66

Introduction

Dans ce chapitre, nous abordons le deuxième sprint du projet SolidMaint, couvrant la période du 12 mars au 2 avril 2026, soit trois semaines de développement consacrées aux modules de gestion métier essentiels de l'application. Nous commencerons par la présentation du backlog du sprint regroupant vingt-trois user stories réparties en quatre epics, portant sur la gestion des clients, des projets et des contrats de maintenance avec leurs SLA associés, suivie de la phase d'analyse et de conception de l'architecture et du modèle de données, puis de la réalisation des interfaces utilisateur, et enfin des tests conduits pour valider les fonctionnalités livrées.

Sprint Backlog

Le tableau ci-dessous présente le Sprint Backlog détaillé avec les descriptions techniques et l'effort estimé pour chaque user story.

ID US	User Story	Description technique	Effort (j)
Epic 5 - Gestion des clients			
US18	En tant qu'administrateur, je souhaite créer un profil client afin de centraliser ses informations.	Formulaire de création avec validation, génération d'identifiant unique, stockage en base	2
US19	En tant qu'administrateur, je souhaite consulter la liste des clients afin de gérer efficacement le portefeuille.	DataTable avec pagination, filtres, recherche, export, indicateurs de statut	2
US20	En tant qu'administrateur, je souhaite consulter le profil détaillé d'un client afin de visualiser toutes ses informations.	Page de détail avec onglets (infos, contrats, projets, historique), graphiques	2
US21	En tant qu'administrateur, je souhaite modifier les informations d'un client afin de maintenir des données à jour.	Interface d'édition, validation, traçabilité, notification conditionnelle	1
US22	En tant qu'administrateur, je souhaite activer et désactiver un compte client afin de contrôler l'accès.	Toggle de statut avec confirmation, vérification des dépendances, notification	1
US23	En tant qu'administrateur, je souhaite ajouter des notes privées sur un client afin de conserver des informations internes.	Éditeur de notes avec date/auteur, stockage en base, visibilité restreinte	1
US24	En tant qu'administrateur, je souhaite imprimer l'annuaire d'un client afin de disposer d'une version papier.	Génération de PDF avec template professionnel, logo, toutes les informations	1
Epic 6 - Gestion des projets			

ID US	User Story	Description technique	Effort (j)
US25	En tant qu'administrateur, je souhaite créer un projet rattaché à un client afin d'organiser les interventions.	Formulaire de création, sélection client, assignation chef de projet, dates	2
US26	En tant qu'administrateur, je souhaite consulter la liste des projets afin de piloter l'activité.	DataTable avec filtres (client, statut, chef de projet), recherche, export	2
US27	En tant qu'administrateur, je souhaite affecter des développeurs à un projet afin de constituer l'équipe.	Interface multi-sélection, vérification disponibilité, notification aux développeurs	2
US28	En tant que membre d'équipe, je souhaite consulter les informations du projet afin de comprendre le contexte.	Page de détail avec infos complètes (client, équipe, contrat, objectifs)	1
Epic 7 - Gestion des contrats			
US29	En tant qu'administrateur, je souhaite créer un contrat de maintenance afin de formaliser les engagements de service avec le client.	Formulaire complet (client, dates, type SLA, cycle facturation), configuration alertes	3
US30	En tant qu'administrateur, je souhaite consulter la liste des contrats afin de suivre les engagements contractuels.	DataTable avec filtres, indicateurs visuels d'expiration, recherche, export	2
US31	En tant qu'administrateur, je souhaite recevoir des alertes avant l'expiration des contrats afin d'anticiper les renouvellements.	Système de cron job, calcul des dates J-30/J-15/J-7, envoi multi-canal (email, in-app, Slack)	3
US32	En tant qu'administrateur, je souhaite archiver un contrat afin de conserver l'historique.	Changement de statut avec confirmation, conservation des données, possibilité de restauration	1
US33	En tant que client, je souhaite demander la résiliation de mon contrat afin d'interrompre le service.	Formulaire de demande avec motif et date, création en base, notification admin	1
US34	En tant qu'administrateur, je souhaite résilier un contrat afin de l'interrompre avant terme.	Interface de résiliation, clôture des tickets ouverts, notification client, archivage	2
US35	En tant qu'administrateur, je souhaite renouveler un contrat afin de prolonger la relation.	Création d'un nouveau contrat à partir de l'existant, lien entre contrats, notification	2

ID US	User Story	Description technique	Effort (j)
US36	En tant que chef de projet, je souhaite consulter les contrats dans un calendrier afin de visualiser les dates d'expiration.	Vue calendrier mensuel/annuel, code couleur selon échéance, export iCal	2
US37	En tant qu'administrateur, je souhaite centraliser les documents d'un contrat afin de les retrouver facilement.	Upload de documents, versioning automatique, association au contrat	2
US38	En tant qu'administrateur, je souhaite configurer les règles SLA globales afin de définir les standards.	Interface de configuration des délais par priorité (MINEUR, MAJEUR, CRITIQUE)	2
Epic 14 - Historique et traçabilité			
US96	En tant qu'administrateur, je souhaite consulter l'historique complet de toutes les actions afin d'assurer la traçabilité.	Interface d'audit avec tous les logs (utilisateur, date, IP, action, entité, valeurs)	2
US97	En tant qu'administrateur, je souhaite exporter les logs d'audit afin de répondre aux exigences de conformité.	Export CSV/JSON avec filtres, signature numérique, chiffrement	2
		Total Sprint 2 :	**42 j**

1 Analyse

1.1 Diagramme des cas d'utilisation du Sprint 2

Cette section présentera le diagramme de cas d'utilisation du deuxième sprint, couvrant la gestion des clients, projets et contrats.

[À compléter avec le diagramme de cas d'utilisation]

1.2 Description textuelle des cas d'utilisation

Cette section détaillera les scénarios principaux et alternatifs pour les cas d'utilisation du Sprint 2.

[À compléter avec les descriptions textuelles]

2 Conception

2.1 Diagrammes de séquence

Cette section présentera les diagrammes de séquence détaillant les interactions pour la gestion des clients, projets et contrats.

[À compléter avec les diagrammes de séquence]

2.2 Modèle de données

Cette section présentera le modèle de données relatif à la gestion des clients, projets, contrats et audit.

[À compléter avec le schéma de base de données]

3 Réalisation

3.1 Architecture technique

Cette section présentera l'architecture technique mise en place pour le Sprint 2, incluant les modules de gestion des clients, projets et contrats.

[À compléter avec les détails d'implémentation]

3.2 Interfaces utilisateur

Cette section présentera les principales interfaces développées durant le Sprint 2.

[À compléter avec les captures d'écran]

4 Tests

4.1 Tests unitaires

Cette section présentera les tests unitaires réalisés pour valider les fonctionnalités du Sprint 2.

[À compléter avec les résultats des tests]

4.2 Tests d'intégration

Cette section présentera les tests d'intégration effectués pour s'assurer du bon fonctionnement global du système.

[À compléter avec les résultats des tests]

Chapitre 5. Sprint 3 : Gestion des Demandes et Tickets

Plan

1 Backlog du Sprint 3 68

2 Analyse 72

3 Conception 73

4 Réalisation 74

5 Tests 75

Introduction

Dans ce chapitre, nous abordons le troisième sprint du projet SolidMaint, couvrant la période du 3 avril au 1er mai 2026, soit quatre semaines de développement consacrées au cœur fonctionnel de l'application, à savoir la gestion des demandes clients et des tickets de maintenance. Nous commencerons par la présentation du backlog du sprint regroupant trente-six user stories, suivie de la phase d'analyse et de conception, puis de la réalisation des interfaces intégrant un tableau Kanban, un suivi du temps de travail, un système de gestion des SLA et des dashboards adaptés à chaque rôle, et enfin des tests conduits pour valider les fonctionnalités livrées.

Sprint Backlog

Le tableau ci-dessous présente le Sprint Backlog détaillé avec les descriptions techniques et l'effort estimé pour chaque user story.

ID US	User Story	Description technique	Effort (j)
Epic 8 - Gestion des demandes			
US39	En tant que client, je souhaite soumettre une demande de maintenance afin de signaler un besoin.	Formulaire de soumission, upload de pièces jointes (max 5), génération numéro unique, confirmation email	2
US40	En tant que chef de projet, je souhaite consulter les demandes en attente afin de les traiter.	Liste filtrée par statut EN_ATTENTE, tri par date, indicateurs de priorité	1
US41	En tant que chef de projet, je souhaite valider une demande afin de la convertir en ticket.	Interface de validation, définition catégorie/priorité, création automatique de ticket lié	2
US42	En tant que chef de projet, je souhaite demander des informations complémentaires afin de clarifier une demande.	Formulaire de questions, changement statut INFOS_REQUISES, relance automatique après 48h	2
US43	En tant que client, je souhaite compléter ma demande afin de fournir les informations manquantes.	Formulaire de réponse, retour automatique au statut EN_ATTENTE	1
US44	En tant que chef de projet, je souhaite requalifier une demande afin de corriger sa catégorie ou sa priorité.	Modification type/priorité avec justification obligatoire, traçabilité	1
US45	En tant qu'utilisateur, je souhaite commenter une demande afin de communiquer sur le contexte.	Éditeur riche, mentions @utilisateur, pièces jointes possibles	2

ID US	User Story	Description technique	Effort (j)
US46	En tant qu'utilisateur, je souhaite consulter l'historique d'une demande afin de suivre son évolution.	Timeline chronologique avec tous les événements, filtres par type, export PDF	1
Epic 9 - Gestion des tickets			
US47	En tant que développeur, je souhaite visualiser mes tickets dans un tableau Kanban afin de gérer mes tâches.	Tableau Kanban avec colonnes (OUVERT, EN_COURS, EN_REVUE, RESOLU, FERME), drag & drop, indicateurs SLA	3
US48	En tant que chef de projet, je souhaite assigner un ticket à un développeur afin de répartir la charge.	Sélection développeur, vérification disponibilité/compétences, alerte surcharge	2
US49	En tant que chef de projet, je souhaite assigner un ticket à plusieurs développeurs afin de collaborer sur des tâches complexes.	Multi-sélection, définition rôles (principal/support), répartition temps estimé	2
US50	En tant que développeur, je souhaite me désassigner d'un ticket afin de libérer ma charge.	Désassignation avec motif obligatoire, retour statut OUVERT, notification chef projet	1
US51	En tant que développeur, je souhaite démarrer un chronomètre sur un ticket afin de tracker mon temps.	Bouton Démarrer, création session de travail, indicateur visuel chrono actif	2
US52	En tant que développeur, je souhaite mettre en pause mon chronomètre afin de gérer les interruptions.	Bouton Pause, enregistrement temps écoulé, possibilité de reprise	1
US53	En tant que développeur, je souhaite arrêter le chronomètre afin de finaliser ma session.	Arrêt avec commentaire optionnel, ajout session à l'historique, notification si dépassement	2
US54	En tant que développeur, je souhaite ajouter manuellement une session de travail afin de corriger des oublis.	Formulaire (date, heures, durée, commentaire), validation sans chevauchement	1
US55	En tant que développeur, je souhaite changer le statut d'un ticket afin de refléter l'avancement.	Workflow avec transitions validées, notification parties prenantes	2
US56	En tant que client, je souhaite valider la résolution d'un ticket afin de confirmer que le problème est résolu.	Interface de validation avec commentaire, fermeture ticket, notification développeur	1

ID US	User Story	Description technique	Effort (j)
US57	En tant que chef de projet, je souhaite rouvrir un ticket fermé afin de traiter un problème récurrent.	Réouverture avec motif, réinitialisation SLA, notification assignés	1
US58	En tant que chef de projet, je souhaite consulter le temps passé par ticket afin d'analyser la productivité.	Affichage temps total et détaillé par développeur, graphiques	2
US59	En tant que développeur, je souhaite signaler un blocage afin d'alerter sur un obstacle.	Formulaire (type, description, impact), notification urgence chef projet, pause SLA	2
US60	En tant que membre d'équipe, je souhaite résoudre un blocage afin de permettre au développeur de reprendre son travail.	Résolution avec commentaire obligatoire, reprise chrono SLA, traçabilité	1
US61	En tant que développeur, je souhaite demander une extension de délai SLA afin de disposer de plus de temps.	Formulaire de demande (durée, justification), soumission pour validation	1
US62	En tant que chef de projet, je souhaite approuver ou rejeter les demandes d'extension SLA.	Liste demandes en attente, approbation/rejet avec commentaire, mise à jour auto SLA	1
US63	En tant qu'utilisateur, je souhaite commenter un ticket avec mentions afin de communiquer efficacement.	Éditeur riche, autocomplétion @utilisateur, distinction internes/externes	2
US64	En tant qu'utilisateur, je souhaite ajouter des pièces jointes aux tickets afin de fournir du contexte.	Upload par drag & drop (images/PDF/logs, max 10MB), prévisualisation	1
US65	En tant que développeur, je souhaite être informé lorsqu'un autre utilisateur modifie le même ticket.	WebSocket temps réel, toast notification, indicateur "X est en train d'éditer"	2
US66	En tant qu'utilisateur, je souhaite consulter l'historique complet d'un ticket afin de comprendre son évolution.	Timeline chronologique inversée, filtres par type, export PDF	1
Epic 17 - Gestion des sous-tâches			
US107	En tant qu'utilisateur, je souhaite créer des sous-tâches pour décomposer un ticket afin de mieux organiser le travail.	Interface de création (titre, description, estimation), calcul progression globale auto	2

ID US	User Story	Description technique	Effort (j)
US108	En tant que chef de projet, je souhaite assigner des sous-tâches à différents développeurs afin de paralléliser le travail.	Assignation indépendante, notifications, suivi temps individuel	2
US109	En tant qu'utilisateur, je souhaite suivre la progression via les sous-tâches afin de visualiser l'avancement détaillé.	Barre de progression, affichage statut chaque sous-tâche, identification blocages	1
US110	En tant que système, je souhaite fermer automatiquement le ticket parent lorsque toutes les sous-tâches sont terminées.	Vérification auto à chaque changement statut, passage RESOLU, notification client	1
Epic 13 - Dashboards			
US89	En tant qu'administrateur, je souhaite consulter un dashboard global afin de piloter le système.	KPIs (utilisateurs actifs, tickets ouverts, taux SLA, CA), graphiques, alertes, filtres période	3
US90	En tant que chef de projet, je souhaite consulter un dashboard d'équipe afin de piloter l'activité.	Vue équipe (charge travail par dev, performance SLA, temps passé vs estimé)	3
US91	En tant que développeur, je souhaite consulter mon dashboard personnel afin d'organiser ma journée.	Tickets assignés avec priorités, chronomètre actif, temps passé, échéances SLA	2
US92	En tant que client, je souhaite consulter mon dashboard afin de suivre mes projets.	Contrats actifs, demandes/tickets en cours, statistiques, documents récents, contacts équipe	2
		Total Sprint 3 :	**60 j**

1 Analyse

1.1 Diagramme des cas d'utilisation du Sprint 3

Cette section présentera le diagramme de cas d'utilisation du troisième sprint, couvrant la gestion des demandes, tickets et dashboards.

[À compléter avec le diagramme de cas d'utilisation]

1.2 Description textuelle des cas d'utilisation

Cette section détaillera les scénarios principaux et alternatifs pour les cas d'utilisation du Sprint 3.

[À compléter avec les descriptions textuelles]

2 Conception

2.1 Diagrammes de séquence

Cette section présentera les diagrammes de séquence détaillant les interactions pour la gestion des demandes, tickets et suivi du temps.

[À compléter avec les diagrammes de séquence]

2.2 Modèle de données

Cette section présentera le modèle de données relatif à la gestion des demandes, tickets, sous-tâches et sessions de travail.

[À compléter avec le schéma de base de données]

3 Réalisation

3.1 Architecture technique

Cette section présentera l'architecture technique mise en place pour le Sprint 3, incluant les modules de gestion des demandes, tickets et dashboards.

[À compléter avec les détails d'implémentation]

3.2 Interfaces utilisateur

Cette section présentera les principales interfaces développées durant le Sprint 3, notamment le tableau Kanban et les dashboards.

[À compléter avec les captures d'écran]

4 Tests

4.1 Tests unitaires

Cette section présentera les tests unitaires réalisés pour valider les fonctionnalités du Sprint 3.

[À compléter avec les résultats des tests]

4.2 Tests d'intégration

Cette section présentera les tests d'intégration effectués pour s'assurer du bon fonctionnement global du système.

[À compléter avec les résultats des tests]

Chapitre 6. Sprint 4 : Collaboration, Documents et IA

Plan

1 Backlog du Sprint 4 78

2 Analyse 82

3 Conception 83

4 Réalisation 84

5 Tests 85

Introduction

Dans ce chapitre, nous abordons le quatrième et dernier sprint du projet SolidMaint, couvrant la période du 2 mai au 31 mai 2026, soit quatre semaines de développement consacrées aux fonctionnalités avancées de collaboration, de gestion documentaire et d'intelligence artificielle. Nous commencerons par la présentation du backlog du sprint regroupant trente-quatre user stories réparties en six epics, suivie de la phase d'analyse et de conception, puis de la réalisation des interfaces intégrant un système de messagerie en temps réel, un module de gestion documentaire avec versioning, un système de notifications multicanaux et un moteur d'intelligence artificielle dédié à la catégorisation automatique des demandes et à la recommandation de solutions, et enfin des tests conduits pour valider les fonctionnalités livrées.

Sprint Backlog

Le tableau ci-dessous présente le Sprint Backlog détaillé avec les descriptions techniques et l'effort estimé pour chaque user story.

ID US	User Story	Description technique	Effort (j)
Epic 10 - Messagerie et collaboration			
US67	En tant qu'utilisateur, je souhaite accéder à une messagerie intégrée afin de communiquer avec mon équipe.	Sidebar avec channels, indicateurs messages non lus, statut présence utilisateurs	2
US68	En tant qu'utilisateur, je souhaite envoyer des messages dans un channel afin de communiquer.	Éditeur riche, mentions @, pièces jointes, livraison via WebSocket	2
US69	En tant qu'utilisateur, je souhaite répondre à un message afin de créer un fil de discussion.	Thread dédié pour réponses, notifications uniquement aux participants	2
US70	En tant qu'utilisateur, je souhaite mentionner des collègues afin d'attirer leur attention.	Autocomplétion @nom, notification push, badge mention non lue	1
US71	En tant qu'utilisateur, je souhaite éditer mes messages afin de corriger des erreurs.	Édition possible dans les 15min, indicateur "(modifié)", historique modifications	1
US72	En tant qu'utilisateur, je souhaite supprimer mes messages afin de retirer du contenu inapproprié.	Suppression avec confirmation, remplacement par "[Message supprimé]", conservation pour audit	1
US73	En tant qu'utilisateur, je souhaite épingler des messages importants afin de les retrouver facilement.	Épinglage (permission requise), limite 10 par channel, possibilité désépinglage	1
US74	En tant que chef de projet, je souhaite voir qui est en ligne dans mon équipe afin de savoir qui je peux solliciter.	Liste membres en ligne, filtrage par projet/équipe, dernière activité	1
US75	En tant qu'utilisateur, je souhaite voir qui est en ligne afin de savoir qui est disponible.	Indicateur statut (EN_LIGNE, ABSENT, NE_PAS_DERANGER), modifiable	1
Epic 11 - Gestion des documents			

ID US	User Story	Description technique	Effort (j)
US76	En tant qu'utilisateur, je souhaite uploader des documents afin de centraliser les fichiers.	Upload drag & drop, association contrat/projet/ticket, métadonnées (titre, description, tags)	2
US77	En tant qu'utilisateur, je souhaite télécharger des documents afin de les consulter hors ligne.	Téléchargement individuel/multiple (ZIP), vérification permissions, enregistrement consultation	1
US78	En tant qu'utilisateur, je souhaite gérer les versions des documents afin d'assurer la traçabilité.	Upload nouvelle version, numérotation auto (v1, v2), commentaire obligatoire	2
US79	En tant qu'utilisateur, je souhaite consulter l'historique des consultations afin de voir qui a accédé au document.	Historique pour traçabilité (qui, quand, durée)	1
US80	En tant qu'utilisateur, je souhaite archiver des documents afin de nettoyer la vue active.	Archivage avec confirmation, conservation données, restauration possible	1
US81	En tant qu'utilisateur, je souhaite être alerté si un fichier identique existe déjà afin d'éviter les doublons.	Calcul hash à l'upload, alerte si doublon, options (remplacer/créer version/annuler)	2
US82	En tant que chef de projet, je souhaite générer automatiquement un PV à partir d'un ticket afin de documenter les interventions.	Génération PDF professionnel (logo, en-tête, description, actions, temps, signature électronique)	3
US83	En tant que client, je souhaite télécharger les PV de mes tickets afin de conserver une trace des interventions.	Accès PV depuis ticket, historique consultable, notification email à génération	1
Epic 12 - Notifications et alertes			
US84	En tant qu'utilisateur, je souhaite recevoir des notifications in-app afin d'être informé en temps réel.	Icône avec compteur, dropdown listant notifications, marquage lu/non lu, accès direct élément	2
US85	En tant qu'utilisateur, je souhaite recevoir des notifications par email afin d'être informé hors connexion.	Préférences fréquence (temps réel/digest), templates professionnels	2
US86	En tant que membre technique, je souhaite recevoir des notifications Slack afin d'être alerté sur mon outil de travail.	Configuration Slack par contrat/projet, webhook URL, événements sélectionnables	2

ID US	User Story	Description technique	Effort (j)
US87	En tant qu'utilisateur, je souhaite recevoir des notifications push navigateur afin d'être alerté même hors onglet.	Demande permission, notifications événements critiques, clic ouvre application	2
US88	En tant qu'utilisateur, je souhaite consulter l'historique de mes notifications afin de retrouver une information.	Liste complète avec filtres (type/statut/période), recherche, pagination	1
Epic 13 - Rapports			
US93	En tant qu'administrateur, je souhaite générer un rapport d'activité mensuel afin de présenter les statistiques.	Rapport (tickets traités, temps passé, respect SLA), export PDF/Excel, graphiques	3
US94	En tant qu'administrateur, je souhaite générer des rapports de conformité SLA afin de justifier les performances.	Rapport taux respect par contrat/période, détail par ticket, calcul pénalités, PDF professionnel	3
US95	En tant que chef de projet, je souhaite générer des rapports d'activité afin de présenter l'avancement.	Rapport par projet/équipe/période (tickets traités, temps passé), graphiques évolution	2
Epic 15 - Versioning avancé			
US98	En tant qu'utilisateur, je souhaite créer une nouvelle version d'un document afin de maintenir l'historique.	Upload avec commentaire obligatoire, numérotation auto (v1.0, v1.1, v2.0), conservation versions	2
US99	En tant qu'utilisateur, je souhaite comparer deux versions d'un document afin de voir les différences.	Sélection deux versions, affichage côte à côte, mise en évidence modifications	2
US100	En tant qu'administrateur, je souhaite voir qui a consulté un document afin de suivre sa diffusion.	Liste consultations (date/heure/utilisateur, durée), statistiques	1
Epic 16 - Assistance intelligente			
US101	En tant que client, je souhaite que le système suggère automatiquement une catégorie pour ma demande.	Analyse titre/description, suggestion catégorie, acceptation/modification possible	3

ID US	User Story	Description technique	Effort (j)
US102	En tant qu'administrateur, je souhaite que le modèle de suggestion s'améliore afin d'offrir des recommandations précises.	Apprentissage des validations/rejets, réentraînement périodique, suivi métriques précision	3
US103	En tant que chef de projet, je souhaite que le système suggère une priorité afin de traiter les demandes efficacement.	Analyse contenu et contexte client, suggestion priorité, ajustable par chef projet	2
US104	En tant que développeur, je souhaite des suggestions de solutions basées sur le passé afin de résoudre plus rapidement.	Identification tickets similaires, suggestion solutions précédentes, acceptation/rejet	3
US105	En tant que développeur, je souhaite évaluer la pertinence des suggestions afin d'améliorer la qualité.	Système notation (utile/pas utile), collecte retours, ajustement modèle	2
US106	En tant qu'administrateur, je souhaite que le système détecte automatiquement les doublons afin d'éviter les demandes redondantes.	Analyse similarité à création, signalement doublons, liaison automatique demandes similaires	2
		Total Sprint 4 :	**64 j**

1 Analyse

1.1 Diagramme des cas d'utilisation du Sprint 4

Cette section présentera le diagramme de cas d'utilisation du quatrième sprint, couvrant la messagerie, gestion documentaire, notifications et IA.

[À compléter avec le diagramme de cas d'utilisation]

1.2 Description textuelle des cas d'utilisation

Cette section détaillera les scénarios principaux et alternatifs pour les cas d'utilisation du Sprint 4.

[À compléter avec les descriptions textuelles]

2 Conception

2.1 Diagrammes de séquence

Cette section présentera les diagrammes de séquence détaillant les interactions pour la messagerie, gestion documentaire et système d'IA.

[À compléter avec les diagrammes de séquence]

2.2 Modèle de données

Cette section présentera le modèle de données relatif à la messagerie, documents, notifications et modèles d'IA.

[À compléter avec le schéma de base de données]

3 Réalisation

3.1 Architecture technique

Cette section présentera l'architecture technique mise en place pour le Sprint 4, incluant les modules de messagerie temps réel, gestion documentaire et IA.

[À compléter avec les détails d'implémentation]

3.2 Interfaces utilisateur

Cette section présentera les principales interfaces développées durant le Sprint 4, notamment la messagerie et le système de notifications.

[À compléter avec les captures d'écran]

4 Tests

4.1 Tests unitaires

Cette section présentera les tests unitaires réalisés pour valider les fonctionnalités du Sprint 4.

[À compléter avec les résultats des tests]

4.2 Tests d'intégration

Cette section présentera les tests d'intégration effectués pour s'assurer du bon fonctionnement global du système.

[À compléter avec les résultats des tests]